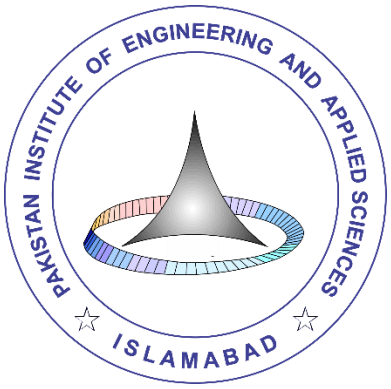
ML-Based Insider Threat Detection for Enterprise Data Exfiltration



by

Ali Raza
(03-3-1-007-2022)
Muhammad Hammad Tahir
(03-3-1-033-2022)
Muhammad Talal Haider
(03-3-1-040-2022)

Supervisor
Dr. Asifullah Khan

Thesis submitted in partial fulfillment of requirements for the Degree of Bachelor of Sciences in Computer and Information Sciences

In

Department of Computer and Information Sciences
Pakistan Institute of Engineering & Applied Sciences

(June, 2026)

Declaration of Originality

I hereby declare that the work contained in this thesis and the intellectual content of this thesis are the product of my own work. This thesis has not been previously published in any form nor does it contain any verbatim of the published resources which could be treated as infringement of the international copyright law.

I also declare that I do understand the terms ‘copyright’ and ‘plagiarism,’ and that in case of any copyright violation or plagiarism found in this work, I will be held fully responsible of the consequences of any such violation.

Signature: _____

Author’s Name: Ali Raza

Signature: _____

Author’s Name: Muhammad Hammad Tahir

Signature: _____

Author’s Name: Muhammad Talal Haider

Date:

Place: PIEAS, Islamabad, Pakistan

Certificate of Approval

Certified that work contained in this thesis titled "**ML-Based Insider Threat Detection For Enterprise Data Exfiltration"** was carried out by **Ali Raza, Muhammad Hammad Tahir, and Muhammad Talal Haider** under my/our supervision and that in my/our opinion, it is fully adequate, in scope and quality, for the degree of BS in Computer and Information Sciences.

Supervisor

Signature: _____

Name: Dr. Asifullah Khan

Head of the Department

Signature: _____

Name: Dr. Hanif Durad

Stamp:

Dedication

This work is dedicated to our families, whose unconditional love, patience, and unwavering support have been the foundation of every achievement.

We also dedicate this thesis to our mentor, Dr. Asifullah Khan, whose guidance illuminated the path of this research.

And to every cybersecurity professional working tirelessly to protect organizations from threats within.

Acknowledgement

We would like to express our profound gratitude to our supervisor, Dr. Asifullah Khan, Professor at the Department of Computer and Information Sciences, PIEAS, for his exceptional guidance, constructive criticism, and continuous encouragement throughout this research journey.

Our sincere appreciation extends to the Pattern Recognition Lab, the Deep Learning Lab at the Center for Mathematical Sciences, and the PIEAS Artificial Intelligence Center (PAIC) for providing the computational resources and collaborative research environment essential for conducting experiments of this scale.

We are grateful to our faculty members at the Department of Computer and Information Sciences for their valuable insights and thought-provoking discussions during our coursework, which contributed significantly to the theoretical foundations of this thesis.

We acknowledge the CERT Division at Carnegie Mellon University's Software Engineering Institute for developing and publicly releasing the CERT Insider Threat Dataset (r6.2), which served as the empirical backbone of this research. Without access to this benchmark dataset, rigorous evaluation of the proposed framework would not have been feasible.

Special thanks to our colleagues and peers at PIEAS who provided feedback, shared ideas, and created an intellectually stimulating environment.

Finally, and most importantly, we owe immeasurable gratitude to our families for their endless patience, emotional support, and encouragement. Their belief in our abilities sustained us through every challenge encountered during this research.

Table of Contents

1. Introduction 12

1.1. Research Background and Motivation 12

1.2. Problem Definition 13

1.3. Limitations of Existing Approaches 14

1.4. Research Objectives 14

1.5. Research Contributions 15

2. Literature Review 17

2.1. Foundations of Insider Threat Research 17

2.2. Rule-Based and Traditional Security Approaches 18

2.3. Machine Learning and UEBA Methods 18

2.4. NLP and Semantic Analysis in Cybersecurity 19

2.5. Multi-Agent and Ensemble Security Systems 20

2.6. Blackboard Architecture in AI Systems 21

2.7. Research Gap Analysis 21

3. Methodology 23

3.1. Dataset Description 23

3.1.1. Dataset Selection Rationale 23

3.1.2. CERT r6.2 Dataset Overview 23

3.1.3. Data Attributes and Log Types 24

3.2. Data Preprocessing and Sessionization 24

3.2.1. Sessionization Framework 24

3.2.2. Feature Engineering 25

3.2.3. Scenario Scope Definition 27
3.2.4. Dataset Split Strategy 28
3.3. Multi-Agent System Architecture 28
3.3.1. System Overview and Shared Blackboard 28
3.3.2. Agent 1 - Contractive Autoencoder 29
3.3.3. Behavioral Normalization Function 31
3.3.4. Agent 2 - Semantic Agent (Fine-Tuned DistilBERT) 31
3.3.5. Agent 3 - Temporal Baseline Agent 32
3.3.6. Agent 4 - Non-Linear Decision Fusion Orchestrator 33
3.4. Hyperparameter Configuration 34
3.5. Implementation Details 35
3.6. Deployment Methodology 36
3.6.1. Deployment Environment Overview 36
3.6.2. Deployment Components 36
3.6.3. ELK Stack Configuration 37
3.6.4. Data Replay and Log Forwarding 38
3.6.5. Inference and Alert Generation 38
4. System Design 39
4.1. Software Engineering Documents 39
4.1.1. Requirements Specifications 39
4.1.2. Use Case Diagram 40
4.1.3. Activity Diagram 41
4.1.4. Sequence Diagram 42
4.2. Overall System Design 43
4.2.1. Data Flow and System Working 43

4.3. Deployment Architecture 44
4.3.1. Windows Client Simulation Environment 44
4.3.2. Ubuntu Monitoring Server 45
4.3.3. Network Configuration 45
4.4. ELK Stack Architecture 46
4.4.1. Filebeat Log Collection 46
4.4.2. Logstash Processing Pipeline 46
4.4.3. Elasticsearch Indexing and Storage 46
4.4.4. Kibana Visualization 46
4.5. Session Aggregation Module 46
4.6. Machine Learning Inference Engine 47
4.7. Dashboard Design 47
4.8. Deployment Validation 48
5. Results 49
5.1. Evaluation Metrics 49
5.2. Individual Agent vs. Multi-Agent System Comparison 50
5.3. Baseline Architecture Comparison 50
5.4. Performance Visualization 51
5.5. Agent-Wise Analysis and Blackboard Communication 52
5.6. Hyperparameter Sensitivity Analysis 52
5.7. Discussion of Results 53
6. Conclusion 55
6.1. Limitations 56
6.2. Future Work 57
References 59

Appendices 62

List of Figures

Figure 1.1: Insider Threat Lifecycle and Its Organizational Impact 13
Figure 3.1: Multi-Agent System Architecture and Blackboard Interaction Flow 29
Figure 3.2: Agent 1 - Global Behavioral Agent (Contractive Autoencoder) 30
Figure 3.3: Agent 2 - Zero-Shot Semantic NLP Scanner (all-mpnet-base-v2) 32
Figure 3.4: Agent 3 - Temporal Baseline Agent 33
Figure 3.5: Agent 4 - Non-Linear Decision Fusion Orchestrator 34
Figure 4.1: Use Case Diagram 40
Figure 4.2: Activity Diagram 41
Figure 4.3: Sequence Diagram 42
Figure 4.4: Deployment Architecture 44
Figure 5.1: ROC-AUC Curves for all agents 51

List of Tables

Table 3-1: Summary of CERT r6.2 Log Types and Extracted Attributes 24
Table 3-2: Session Feature Description 27
Table 3-3: Experimental Hyperparameter Configuration 35
Table 3-4: Deployment Component Summary 37
Table 3-5: ELK Stack Software Versions and Configurations 38
Table 4-1: System Layer Descriptions 43
Table 5-1: Individual Agent and Orchestrator Performance at P_{80} SOC Triage Budget 50
Table 5-2: ROC-AUC Comparison Across Fusion Strategies 51
Table 5-3: Hyperparameter Search Space and Optimal Values 53

Abstract

Inside threats are a serious threat in today's enterprise environment. Malicious users are using legitimate logins to move around the organization, evade traditional Intrusion Detection System (IDS) and quietly steal incredibly sensitive information. User and Entity Behavior Analytics (UEBA) for machine learning has evolved to the current state of the art which only considers the numerical frequency of events in comparison to Static, global thresholds, does not take into consideration semantic intent in unstructured communications, does not share inter-agent context, and does not capture exfiltration behavior from the routine high-volume activity of legitimate users. This thesis presents a novel fully unsupervised Multi-Agent System (MAS) with a Shared Blackboard communication bus with a bidirectional consensus feedback loop. Using the CERT Insider Threat Dataset (r6.2), we introduce a collaborative four-agent architecture: (1) a Global Behavioral Agent (Contractive Autoencoder) integrating reconstruction error with a Behavioral Normalization function to separate exfiltration signals from routine volume noise; (2) a Zero-Shot Semantic NLP Scanner (all-mpnet-base-v2) computing cosine similarity differentials against natural language threat anchors without any domain fine-tuning; (3) a Severity-Weighted Temporal Drift Engine computing domain-prioritized Z-score drift against personalized user baselines; and (4) a Consensus Fusion Orchestrator that executes a recalibration feedback loop before producing rank-normalized ensemble threat rankings. The integrated Behavioral Normalization function addresses the volume-induced false positive problem that afflicts user environments in production Security Operations Centers (SOC). The system was deployed in a dual virtual machine environment comprising a Windows VM for log generation and forwarding via Filebeat, and an Ubuntu VM hosting a full Elasticsearch, Logstash, and Kibana (ELK) Stack integrated with a custom ML Inference Engine, demonstrating practical viability for real-world enterprise security operations.

1. Introduction

Over the last ten years, enterprise security has drastically changed. Organizations have spent many resources on perimeter security like next gen firewalls, signature intrusion detection systems, endpoint protection etc., but one threat is continually compromising security posture which is threat from within. Insider threats are among the most complex, expensive and hard to detect types of cyber incidents that organizations face today. Insiders have access, knowledge and trust within the organization that enables them to steal sensitive data without raising the usual suspicions of outsider infiltration.

1.1. Research Background and Motivation

Modern enterprises are undergoing digital transformation at an increasing pace, supported by hybrid working models and drastically reimagined, highly connected and collaborative infrastructures across the total enterprise security perimeter. Traditional perimeter defenses have come to the fore, making it easier to protect from the outside, but organizations still have to defend against potential vectors of attack from the inside the trusted network. In today's digital age, data is the most valuable resource, and safeguarding it is a regulatory requirement with some of the strictest compliance standards like GDPR and HIPAA. Protecting against this internal compromise has thus become a top priority for both cybersecurity study and enterprise risk management.

According to the 2022 Ponemon Institute Cost of Insider Threats Global Report, insider threat incidents have risen by 44% during the last two years and the average cost of insider threats program has surpassed USD 15.4 million a year across the organizations [1]. The 2023 Verizon Data Breach Investigations Report similarly highlights insider incidents as a persistent and growing category of confirmed data breaches across industries [21]. The numbers highlight a sobering truth that the adversary is already within the line of sight, and has a legitimate credential and a deep understanding of the organization's systems and processes.

The motivation of this research is to understand that current machine-learning based security solutions are solving security problems using data sources in isolation, with mathematically

rigid linear combinations to fuse data outputs, ignoring the complex and non-linear relationships between multi-modal threat signals. A malicious insider launching a highly sophisticated data exfiltration campaign takes full advantage of this limitation by planning low volume movement of data - all individual moves are not likely to trigger the (low-level) alarms, but the (high-level) pattern through the behavioral, semantic and temporal dimensions is clear and imminent.

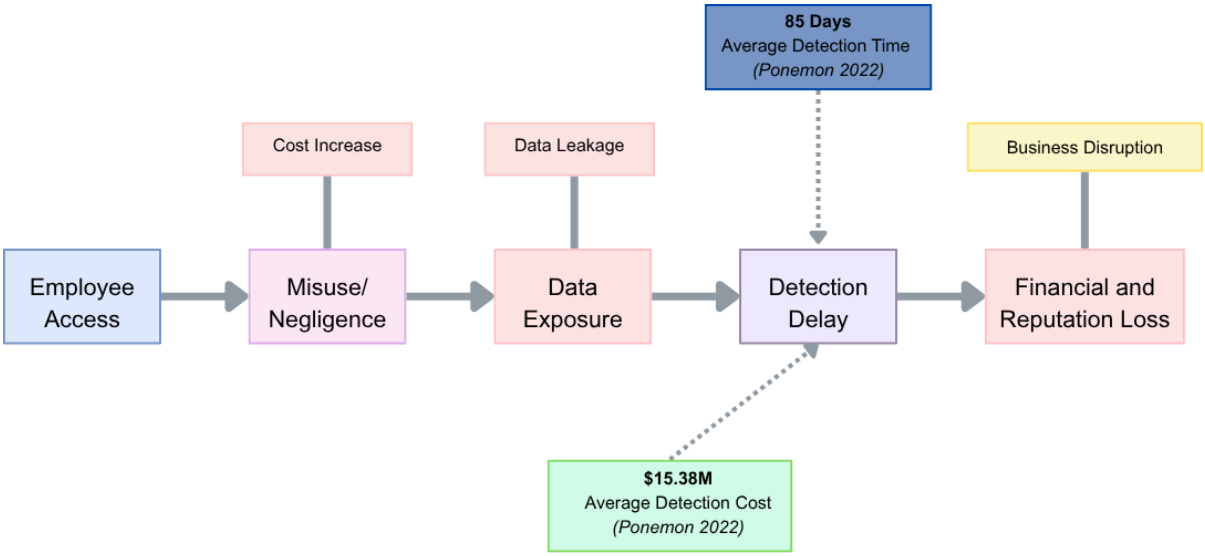


Figure 1.1: Insider Threat Lifecycle and Its Organizational Impact

1.2. Problem Definition

A threat from a trusted or authorized employee, contractor or business partner is a completely different and very one-sided type of threat. These actors have the legitimate access, system knowledge and physical proximity to affect the confidentiality, integrity or availability of the data without the need to penetrate external security perimeters too much. Data Exfiltration is the most damaging of the different types, directly threatening the survival of the organization and its protection of the intellectual property.

Data exfiltration is when a co-worker accesses internal company information or source code, customer information, or any regulated information and transfers it to an outside cloud, personal webmail, or removable media device without the company's permission. Detecting data exfiltration is an operational challenge, as the actions, copying files, sending emails with attachments, conducting web searches, are essentially the same as normal daily activities. The malicious insider, however, has no known cryptographic signature, uses regular office products

and legitimate credentials, making the use of signature-based techniques completely impractical.

The even greater class imbalance found in real enterprise conditions further compounds the problem. If there is a genuine insider threat event, it appears in a much lower proportion to the tremendous amount of legitimate user activity. The ratio of benign to malicious sessions in the benchmark CERT Insider Threat Dataset (r6.2) used in this research, spanning 153,307 test sessions with 16 confirmed breach sessions, reflects these real-world SOC conditions.

1.3. Limitations of Existing Approaches

Conventional security solutions such as rule-based Data Loss Prevention (DLP) products were built on rigid, pre-established assumptions or heuristics, either of which can severely hinder user productivity or produce too many false-positive alerts. Constantly misguided alerts lead to severe ‘alert fatigue’ in Security Operations Centers (SOC), often causing true and subtle threats to go unnoticed.

The landscape of cybersecurity has increasingly been moving towards UEBA with powerful machine learning. There is however a serious architectural problem with current ML-based UEBA systems, (structured) logs are analyzed independently and scores are combined with fixed linear combinations, while ignoring any non-linear relationship between various types of behavioral evidence that might be present in them. Moreover, volumetric thresholds ignore entirely ‘low and slow' attacks, whereby the user takes small portions of data over a long period of time, where no single threshold will be violated but as a matter of the trend, the activity poses an unmistakable risk.

The lack of semantic context in the current framework is yet another serious shortcoming since, regardless of volume, an action such as transferring 50MB of data from one location to another does not imply any malicious intent on its own. However, if at the same time there are signs in the email message indicating that the user may be applying for jobs elsewhere, then this becomes millions of times more suspicious. Current systems do not provide for any form of communication between the agents involved.

1.4. Research Objectives

This research is guided by the following primary objectives:

- To develop and deploy a new Multi-Agent System (MAS) to be able to break down the insider threat detection problem into specialized, collaborative analytical domains that seek to address different behavioral modalities.
- To implement a Shared Blackboard architecture where the agents that process the behavior can dynamically and timely share their context with semantic and temporal agents, modulating the sensitivity of these semantic and temporal agents when they meet anomalies.
- To deploy a zero-shot semantic NLP scanner using pre-trained sentence embeddings (all-mpnet-base-v2) to evaluate behavioral intent through cosine similarity differentials against natural language threat and benign anchor embeddings, without requiring any domain-specific fine-tuning.
- To design a Behavioral Normalization (BN) function integrated in all agents that quantifies the net contrast between sensitive exfiltration indicators and routine activity volume, directly suppressing false positives generated by legitimate users.
- To execute a non-linear decision fusion mechanism between heterogeneous agents output scales, with complex multi-dimensional threat boundaries, by using an Isolation Forest orchestrator.
- To test the proposed framework with the CERT Insider Threat Dataset (r6.2) on four data exfiltration scenarios, against stand-alone agents and a linear fusion baselines.
- To design and test an end-to-end enterprise deployment pipeline to integrate the proposed MAS with an ELK Stack (Elasticsearch, Logstash, Kibana) infrastructure for log ingestion, storage and threat visualization.

1.5. Research Contributions

The primary contributions of this research to the field of cybersecurity and machine learning are as follows:

- The “4-Agent MAS Architecture with Blackboard Communication” is a new approach to move from opaque, monolithic models towards a modular framework. The Shared Black board (message bus) allowing dynamic inter-agent communication is the core innovation: evidence flags can be posted by upstream agents which change the sensitivity of downstream agents in real time.

- Zero-Shot Semantic NLP Scanner: Deploying all-mpnet-base-v2 sentence embeddings to compute a normalized cosine similarity differential between session narratives and natural language threat/benign anchor embeddings, enabling intent-aware scoring without any domain fine-tuning. Scores are dynamically boosted by consensus multipliers from the Blackboard feedback channel when mutual high-risk consensus is detected.
- Behavioral Normalization (BN) Function: A novel inter agent normalization function that computes the net contrast between weighted exfiltration signal features (large emails, file opens, cloud uploads) and weighted routine noise features (total emails sent, USB activity), integrated into the anomaly scoring of all three detection agents to isolate genuine threats from legitimate high-volume users.
- Unsupervised Consensus Feedback Loop: A bidirectional Blackboard in which the Orchestrator interrogates upstream flags and writes recalibration parameters ($\Delta\beta = +0.25$, $\Delta\sigma = -0.15$) back to the bus when mutual high-risk consensus is detected ($Flag_1 \cap Flag_2$), dynamically recalibrates Agent 1 threshold sensitivity and Agent 2 boost multiplier before final ensemble scoring.
- Personalized Temporal Baselining: An independent agent computing the RMS-aggregated Z-scores has been integrated to allow correct identification of low and slow micro-deviations specific for each user, addressing a critical gap in global approaches based on thresholds.
- Non-Linear Decision Fusion: How to replace traditional linear weighted sums with an Isolation Forest Orchestrator for complex multi-dimensional threat signatures with no outlier washout, even when applying continuous scores alongside of discrete Blackboard flags.
- End-to-End Deployment Pipeline: Enterprise deployment architecture that is fully operational with dual VMs (Windows and Ubuntu), Filebeat for log forwarding, ELK stack (Elastic Search, Logstash and Kibana) for logging and visualization, along with an ML Inference Engine for threat detection.

2. Literature Review

There have been great advancements in insider threat detection from simple audit methods to complex behavioral analysis by application of machine learning techniques in the last two decades. The chapter critically analyzes the development of various approaches used in addressing the issue of insider threats through the classification of methods into rule-based systems, UEBA, NLP, and MAS techniques, ending with an analysis of the identified research gaps.

2.1. Foundations of Insider Threat Research

The concept of insider threat detection as a distinct field of research received a much-needed boost in momentum since the publication of the Computer Emergency Response Team (CERT) Guide to Insider Threats by Cappelli, Moore, and Trzeciak [2]. Initial studies involving taxonomies and psychological profiling looked at how various behaviors, like grievances, financial problems and drastic shifts in work hours, were related to malicious conduct. Insider threats are categorized as financial gain or revenge, negligent insider, and compromised insiders whose credentials have been stolen by an outsider.

In a comprehensive investigation on the most frequently used taxonomies of the insider threat, analysis methods, models, and countermeasures, Homoliak et al. [3] identified 32 types of insider threats and categorized over 500 insider threat case studies available in literature. This contributed to recognizing the multi-facetted nature of insider threats as well as the requirement for multi-modal insider threat detection approaches. Additionally, Le and Zincir-Heywood [4] conducted research on insider threat detection approaches based on anomalies and concluded that the usage of only one behavior data module in the detection system was not sufficient.

The Nurse et al. study [5] put forward an analytical model to explain insider threat attacks and identified three attack components: capabilities of the attacker, predisposition of the attacker, and the opportunity for attack. The understanding of these attack components is critical in

designing any system capable of detecting both volumetric anomalies and behavioral anomalies that occur before exfiltration.

2.2. Rule-Based and Traditional Security Approaches

The first technical responses to counter the occurrence of insider threats were developed around pre-known signatures and pre-made Boolean logic, assisted by a rule-based Expert Systems platform and a Security Information and Event Management (SIEM) platform. The "canonical" implementation of this paradigm is a Data Loss Prevention (DLP) system, which could block an outgoing e-mail message that exceeds the 50 MB limit, or has regular expressions that match the Social Security Number or the pattern of a credit card.

These systems work well in catching obvious breaches of policy, but they're really "static", and require manual reconfiguration to be capable of adapting. According to Hassan et al. [6] false positive rates can be as high as 99% and SIEMs can produce hundreds to thousands of alerts per day in large enterprises. That's the driving force behind the malicious insider, encryption of stolen information, renaming it to evade detection or knowing how to exfiltrate, which is done at just below the alerting level, known as ‘low and slow'.

As discussed by Mutawa [7] the trade off between the precision and the recall is natural to a rule-based system, the tighter sets the rules the more false positives will be produced, but the tighter sets the rules, the more false negative will be produced and the looser sets the rules the more alert fatigue will occur. This simple rule prompted the creation of detection methods based on learning.

2.3. Machine Learning and UEBA Methods

Static rules being limited, the cyber security sector moved to UEBA powered by machine learning. Essentially, the idea of User and Entity Behavior Analytics (UEBA) lies in creating a statistical model of normal behavior of a particular user/entity and detecting deviations from that behavior as signs of danger. Basic UEBA software included Principal Component Analysis (PCA), Support Vector Machines (SVMs), and Random Forests (RFs). Sarhan and Altwaijry [8] showed that tree-based ensembles can effectively classify log events that deviate from the normal pattern of events in the CERT dataset with satisfactory performance for high-volume deviations.

Several clustering methods like K-Means and DBSCAN were used to classify users that have comparable behavioral roles and then compute relative anomaly scoring in the clusters. Chattopadhyay et al. [9] used the CERT dataset to test the effectiveness of clustering methods to detect behavioral outliers that can be classified into roles, using only the unlabeled data.

Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Autoencoders have been very useful in the field, by modeling complex non-linear relationships over time in sequential behavioral data. Tuor et al. [10] used LSTM-based unsupervised anomaly detection on structured log streams proving it outperforms classical statistical approaches. In [11] Chen et al. used deep autoencoders to model the reconstruction manifold of normal behavior, marking sessions that deviated from the normal manifold as outlying. Ali et al. [12] pushed this field forward by adding behavioral analytics and deep evidential clustering to the mix for real time insider threat detection.

However, despite all these developments, one important feature is missing from all UEBA solutions: They rely heavily on numerical feature vectors extracted from structured logs, and do not take into account the meaning of the communications and interactions with files. Moreover, most designs ignore the fact that the overall threshold is applied to everyone, meaning that they are unaware of an individual user baseline and don't account for low-and-slow campaigns operating on that baseline.

2.4. NLP and Semantic Analysis in Cybersecurity

One of the drawbacks of volumetric UEBA is that it is intent-blind. Someone moving 100 MB of data to an outside server could be backing up or stealing intellectual property. The difference is in the meaning of the data being moved and associated messages. A sentiment analysis approach has also been shown to be useful by Taylor et al. [13] who used a lexicon-based sentiment analysis approach to analyze senti-negative and job-seeking content and competitor mentions in corporate email communications for the purpose of detecting malicious insiders.

Since the introduction of transformer-based architectures such as BERT [14] which allows deep semantic context understanding in textual data, the volume of available information has significantly grown. The researchers then used BERT embeddings for the analysis of the HTTP URL strings, file names, and the subject lines of emails to identify unauthorized access to sensitive projects. The deep learning sentiment analysis of corporate email corpora could detect

negative behaviors as potential indicators of future exfiltration as early as two weeks beforehand, as shown by Zhang et al. [15].

Pre-trained sentence embedding models such as all-mpnet-base-v2 offer an alternative to fine tuned classifiers for zero shot semantic evaluation. By giving natural language into a dense 768-dimensional semantic space, these models perform cosine similarity computation between arbitrary text pairs without requiring labeled training data. Reimers and Gurevych [14] demonstrated that sentence level embeddings from models trained with contrastive objectives that outperforms token level BERT embeddings for semantic similarity tasks, making them well-suited for intent scoring via similarity to predefined anchor texts. The challenge for security architectures, however, is integrating high-dimensional semantic signals with low dimensional numerical behavioral features in a dynamic, real-time detection pipeline that is exactly what the Shared Blackboard architecture addresses.

2.5. Multi-Agent and Ensemble Security Systems

Cybersecurity architectures have begun to turn to Multi-Agent Systems (MAS) to cope with the heterogeneity and complexity of network data from modern enterprises, spreading out the analytical load among a number of specialized autonomous agents that come together to deliver detection capabilities that exceed any individual agent. MAS architectures showed potential when applied to Network Intrusion Detection Systems (NIDS) [16] to correlate the various streams of information from the network, host, and application.

Eldardiry et al. [17] proposed a multi-domain information fusion method for insider threat detection that includes network activity, physical access, and psychological aspects. This study, however, relied upon basic voting schemes in the fusion layer where agents are unable to update each others' sensitivity to their upstream input due to the lack of communication with other agents.

A classic example of combining pattern classifiers is provided by Kuncheva [18] in her pioneering work on combining anomaly detectors running on completely orthogonal mathematical scales (such as reconstruction error from an autoencoder, probability score from a language model, Z score from a statistical baseline agent) with simple linear weighted sums is mathematically insufficient. Linear fusion is actually susceptible to ‘outlier washout’, a single agent with a very high score will be ‘washed away’ by a number of agents reporting

near baseline scores. Due to its ability to conduct outlier detection in high-dimensional data with power, non-linear geodesic methods, such as the Isolation Forest [19] proposed by Liu et al., are natural candidates for multi-modal evidence fusion for outlier detection.

2.6. Blackboard Architecture in AI Systems

The Blackboard architecture is a classic problem-solving paradigm in the field of artificial intelligence, first introduced to handle the context of complex and distributed problem-solving, where there is no single expert to solve the whole problem. The idea of a Blackboard system is a system in which there are multiple specialized knowledge sources (agents), where agents indirectly communicate with each other by posting observations and reading the evidence that has been shared by the other agents in the system and not tightly coupled to other agents.

The Blackboard architecture is a classic AI problem-solving paradigm first formalized by Nii [29], in which specialized knowledge sources cooperate by reading and writing to a shared global data store rather than communicating directly. Its key benefit for security systems is decoupling agents can be added, removed, or retrained independently. Hofmeyr and Forrest [20] demonstrated a related distributed detection approach for network security, where agents independently contribute evidence to a shared immunological recognition space. A principle directly analogous to the Blackboard-based evidence aggregation in the proposed MAS. The other major benefit of the Blackboard approach is that the communication topology is inherently scalable and decoupling: the agents can be added, removed or changed without changing the communication topology. This is a very important characteristic of the Blackboard for insider threat detection: upstream behavioral evidence can dynamically shift downstream semantic evidence decision boundary as though in the workflow of a human SOC analyst, who would typically focus on language during the early stages of an investigation when there is a suspicious increase in volume.

2.7. Research Gap Analysis

Based on the above literature review, three crucial research gaps have been discovered which motivate the development of the proposed system as follows:

- Lack of Dynamic Inter-Agent Context Sharing: Current ensemble models test the different modalities separately, and only exchange information at the last fusion level. No architectural support for upstream anomaly signals to dynamically alter the

sensitivity or decision thresholds of agents downstream so as to optimally synthesize multi-modal evidence.

- Volumetric and behavioral models are both fundamentally oblivious of the semantic intent of communications and file accesses and see no difference between a large "normal" back up and a large data theft. There are some NLP approaches but they will do it outside of the behavioral agents and they will not include the mechanisms in order to be aware of the contexts.
- Linear Fusion Inadequacy: Fusing multiple mathematical characteristics, such as autoencoder reconstruction scores, LLM probability scores and statistical Z-scores, by linear weighted sum is inadequate to describe the non-linear, multi-modal decision boundaries of multi-modal insider threat signatures. Existing architectures lack the means of non-linear fusion mechanism for this mixed feature space.

The designed Multi-Agent System based on Shared Blackboard successfully tackles all the three gaps discovered by employing unique architecture, communication between agents, and non-linear Isolation Forests.

3. Methodology

The methodology employed in this research is elaborated upon in detail within this section. The methodology details include data set description and pre-processing pipeline, sessionization scheme for feature generation, architecture of each one of the four agents included within the multi-agent system and mathematical equations governing their functioning, and implementation schema validating its feasibility as an effective and practical solution.

3.1. Dataset Description

3.1.1. Dataset Selection Rationale

Considering the highly sensitive nature and serious legal issues involved with insider threat situations, actual insider threat data sets are not readily available to researchers. For the purpose of replication and comparing results with others who have carried out similar work, the CERT Insider Threat Dataset (Release 6.2) has been used. This dataset is developed and released by the CERT Division of Software Engineering Institute of Carnegie Mellon University.

3.1.2. CERT r6.2 Dataset Overview

The CERT r6.2 dataset is a vast simulation of a realistic organization setting over a period of 18 months. This data simulation models the actions of 4,000 synthetic employees through all significant digital channels, generating tens of thousands of logs for each simulated scenario. This includes both random simulations of legitimate actions and malicious behavioral activities simulated based on real-life examples.

Among other features, the strength of this data set lies in the presence of ground truth that is stored in insiders.csv. For this particular research project, we used the test data set comprising 153,291 benign and 16 malicious sessions.

3.1.3. Data Attributes and Log Types

Multi-sourced detailed logs are contained in the five separate CSV files of the CERT r6.2 dataset. Table 3.1 gives an overview of all types of logs, their attributes, and the importance of each to detecting data exfiltration.

Log Type	Key Attributes	Relevance to Data Exfiltration
Logon	User ID, Date/Time, Activity (Logon/Logoff), PC ID	Determines session boundaries; identifies after-hours physical presence.
Email	User, Date/Time, To, From, Size, Attachments, Content	Tracks external communication volume and linguistic intent; large external attachments are primary exfiltration vectors.
HTTP	User, Date/Time, URL, Content	Identifies access to unauthorized cloud storage; URL semantics reveal intent.
File	User, Date/Time, Filename, Content, Activity	Monitors interactions with sensitive intellectual property; file-to-USB transfers are direct exfiltration evidence.
Device	User, Date/Time, Activity (Connect/Disconnect)	Tracks physical exfiltration vectors; USB device connects correlating with file copies constitute strong evidence.

Table 3-1: Summary of CERT r6.2 Log Types and Extracted Attributes

3.2. Data Preprocessing and Sessionization

3.2.1. Sessionization Framework

Log lines in an unorganized state, gathered from five separate sources, tend to pose challenges for algorithms to contextualize properly. Individual log events do not possess the proper time frame and behavior needed to differentiate between malicious patterns and normal behavior.

In order to solve the problem, log lines from the five different sources are combined into a coherent User Session.

Each session is defined in terms of the time period between the Logon event and the Logoff event of the specific user. Session duration of 8 hours as default limit could be imposed in order to address the problem associated with logging process being incomplete most of the time due to various reasons like rebooting or lack of connectivity. This will ensure that there is no accidental merging of two different days.

3.2.2. Feature Engineering

Each session contains 31-dimensional numerical feature vector that describes login activity, USB usage, file operations, emails data, web and cloud activity, aggregate activity ratios, and 7 historical 30-day rolling Z-score features capturing short-term behavioral shifts relative to each user's recent habits. Agent 2 also constructs a natural language text summary from session features for zero-shot semantic embedding; this summary is generated programmatically from feature thresholds rather than from raw textual log content. Table 3.2 provides details on all features extracted.

Feature Name	Type	Description
duration	Numerical	Total session duration in minutes.
is_weekend	Binary	Indicator for sessions occurring on weekends.
is_after_hour	Binary	Indicator for sessions outside standard working hours.
emails_count	Numerical	Total number of emails sent during the session.
ext_emails_count	Numerical	Number of emails sent to external non-corporate addresses.
attachments_count	Numerical	Number of email attachments sent externally.
total_email_size	Numerical	Aggregate size in bytes of all external email attachments.

http_count	Numerical	Total number of HTTP requests during the session.
cloud_uploads_count	Numerical	Number of requests to known cloud storage domains.
usb_connects	Numerical	Number of USB device connection events.
file_copies	Numerical	Total number of file copy operations performed.
file_to_usb	Numerical	Number of file copy operations directed to a USB device.
usb_after_hours	Numerical	Count of USB connection events outside normal working hours
file_copy	Numerical	Count of file copy/duplicate operations
file_after_hours	Numerical	Count of file operations outside normal working hours
email_to_self_external	Numerical	Count of emails sent to personal external addresses matching user name/ID
email_after_hours	Numerical	Count of emails sent outside normal working hours
http_after_hours	Numerical	Count of HTTP requests outside normal working hours
http_cloud_and_leak_sites	Numerical	Visits to cloud storage or whistleblowing sites (Dropbox, Mega.nz, Wikileaks, Pastebin)
logon_after_hours	Numerical	Count of logon events outside normal hours (before 07:00 or after 19:00)
logon_weekend	Numerical	Count of logon events occurring on Saturday or Sunday

session_duration_hours	Numerical	Total active workday span (last logoff hour − first logon hour)
after_hours_ratio	Numerical	Proportion of total activity occurring after hours
total_events	Numerical	Sum of all primary activity counts
z_logon_after_hours	Numerical	30-day rolling Z-score of after-hours logins
z_http_cloud_and_leak_sites	Numerical	30-day rolling Z-score of cloud/leak site visits
z_email_sent_external	Numerical	30-day rolling Z-score of external email volume
z_usb_connects	Numerical	30-day rolling Z-score of USB usage
z_file_to_removable	Numerical	30-day rolling Z-score of removable media file transfers
z_file_copy	Numerical	30-day rolling Z-score of mass file copying
z_after_hours_ratio	Numerical	30-day rolling Z-score of overall after-hours work proportion

Table 3-2: Session Feature Description

Label assigned to a session is a two-level ground truth: “1” if the time window of the session overlaps with any recorded threat events in insiders.csv for the respective user; “0” otherwise.

3.2.3. Scenario Scope Definition

The research is limited to Data Exfiltration only. Scenario 3 – IT Sabotage – has been explicitly excluded because the signatures associated with it depend on privilege escalation and disruption and not on data theft. The scenarios used are:

- Scenario 1: An insider gains access to company systems after hours, uses removable media and loads sensitive information to high-risk external locations prior to leaving the company utterly. This scenario brings about high anomaly signals both in volume and time.

- Scenario 2 is for a worker showing dissatisfaction with the job by looking at employment portals. They use removable media at a much higher rate before they resign than their normal history. Strong deviation signals in time occur in this situation.
- Scenario 4: An insider accesses the workstation of colleagues without their knowledge, looks for sensitive documents and sends them via a personal email address without their knowledge over a long term, typically over several months. This is the very hard to detect low and slow attack vector, which is the most difficult for volumetric detectors.
- Scenario 5: A disgruntled employee uploads important business files to an unsecure personal cloud storage solution (Dropbox) and potentially tries to earn personal profit from using intellectual property. This situation creates ample semantic clues in the form of URLs and content.

3.2.4. Dataset Split Strategy

The time split strategy is used instead of using the random training-testing split, since using the latter may lead to data leakage from future sessions affecting the calculation of baselines during the past ones, which is important for user profiling by Agent 3. Data is partitioned by calendar date to simulate real world deployment and prevent temporal leakage. The Training Set (≤ 2010-12-31) contains 300,000+ benign sessions used to fit the CAE and Temporal baselines. The Validation Set (2011-01-01 to 2011-03-31) is used exclusively for threshold tuning and hyperparameter selection. The Test Set (> 2011-03-31) comprises 153,307 sessions in which 153,291 are benign and 16 are confirmed malicious, they are used for final operational evaluation under the P_{80} triage budget.

3.3. Multi-Agent System Architecture

3.3.1. System Overview and Shared Blackboard

Traditional ensembles analyze models in parallel and can only interact at the very end in the fusion layer, where the outputs of the models are combined using fixed and predefined weighting approaches. This approach is entirely inadequate for multi-modal insider threat detection since it does not allow upstream agents' output to dynamically impact the downstream model's sensitivity to the threat signals beforehand.

This problem is solved in the new architecture by using a central message bus called the Shared Blackboard. The architecture allows each of the four agents to use the shared blackboard while

working. In addition to holding continuous anomaly scores received from each of the agents, the Blackboard also holds binary indicators whenever a threshold is exceeded. The flow includes the following:

The Blackboard operates in two passes per session evaluation cycle:

Pass 1: Independent Anomaly Scoring: Agent 1 (CAE) computes $S_1(x)$ and writes a high_flag if the composite score exceeds the dynamic threshold τ_1. Agent 2 (Zero-Shot NLP Scanner) computes the cosine similarity differential ΔS_nlp, applies BN(x), writes S_2 and $Flag_2$. Agent 3 (Temporal Drift Engine) computes severity-weighted Z-score drift D(x,u), applies BN(x), and writes S_3. All three agents operate independently in this pass.

Pass 2: Consensus Recalibration and Fusion: Agent 4 (Orchestrator) reads $Flag_1$ and $Flag_2$. When mutual consensus is detected ($Flag_1 \cap Flag_2$), it writes recalibration parameters $\Delta\beta_feedback = +0.25$ and $\Delta\sigma_feedback = -0.15$ back to the Blackboard. Agents 1 and 2 apply these adjustments to produce recalibrated scores before the Orchestrator computes the final rank-normalized fusion $R_final = 0.45 \cdot R(S_2) + 0.35 \cdot R(S_1) + 0.20 \cdot R(S_3)$

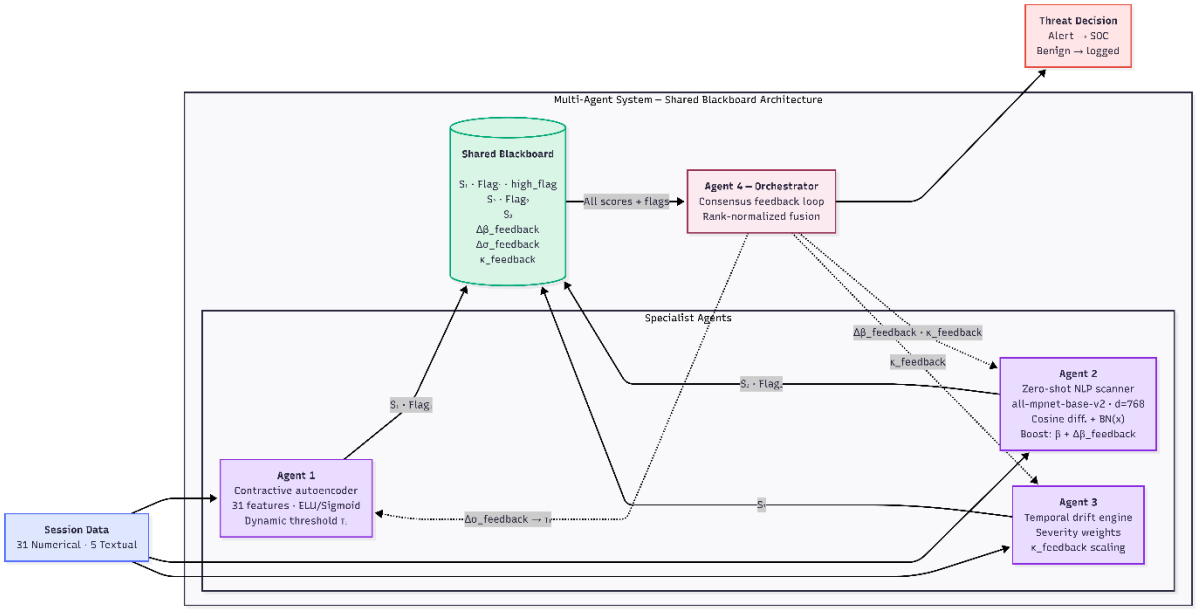


Figure 3.1: Multi-Agent System Architecture and Blackboard Interaction Flow

3.3.2. Agent 1 - Contractive Autoencoder

Agent 1 acts as the main volumetric anomaly detector by comparing the numerical structures of each session against the statistical manifold of normal enterprise behavior learnt for the

whole user base. Contractive Autoencoder (CAE) will be implemented using PyTorch framework with a symmetric encoder-decoder design. The 31-dimensional input vector (12 numerical session features plus 19 engineered behavioral indicators) is mapped through hidden layers [31, 32, 16] to an 8-dimensional bottleneck latent code and reconstructed in reverse [8, 16, 32, 31]. All hidden layers use Exponential Linear Unit (ELU) activation; the bottleneck layer uses a bounded Sigmoid activation to ensure stable Jacobian computation.

The distinguishing element of the CAE model is the use of a contraction term in addition to MSE loss in the form of a Jacobian matrix. The training criterion is:

$$L_{CAE} = L_{MSE} + \lambda \times \left\| J_{f(x)} \right\|_F^2 \tag{3.1}$$

where $\left\| J_{f(x)} \right\|_F^2$ is the Frobenius norm of the Jacobian of the encoding function f(x) with regard to its input, and λ is the hyperparameter weight of the penalty term whose value is chosen as 1×10^{-4}. The Frobenius norm is computed analytically over the bottleneck units as:

$$\left\| J_{f(x)} \right\|_F^2 = \left[\Sigma_{\{j\} h_j} \cdot \left(1 - h_j\right)^2 \right] \cdot \|W\|_F^2 \tag{3.2}$$

This penalty constrains the encoder to learn only the most invariant aspects of normal behavior, suppressing sensitivity to irrelevant local input perturbations.

$$S_1(x) = \left(\frac{1}{d}\right) \|\hat{x} - x\|_2^2 + BN(x) \tag{3.3}$$

The dynamic detection threshold is recalibrated by the Orchestrator feedback channel:

$$\tau_1 = \mu_{train} + \left(\sigma_{base} + \Delta\sigma_{feedback}\right) \times std_{train} \tag{3.4}$$

where $\Delta\sigma_{feedback}$ is written by the Orchestrator to the Blackboard when mutual high-risk consensus is detected, tightens the threshold to increase sensitivity. Sessions above τ_1 trigger a high_flag on the Shared Blackboard, notifying downstream agents of data exfiltration evidence.



Figure 3.2: Agent 1 - Global Behavioral Agent (Contractive Autoencoder)

3.3.3. Behavioral Normalization Function

A core vulnerability of standard anomaly detectors in SOC environments is volume-induced false positives: legitimate power users generate high volumes of sensitive indicators as part of routine workflows. To isolate genuine exfiltration behavior, a Behavioral Normalization (BN) function is integrated into the anomaly scoring of all three detection agents. BN quantifies the net contrast between exfiltration signal features and routine noise features:

$$BN(x) = \Sigma_{\{j \in F_{signal}\}} w_j \times x_j - \Sigma_{\{k \in F_{noise}\}} v_k \times x_k \tag{3.5}$$

With the following feature weights applied to standardized inputs:

$$\begin{aligned} BN(x) = \big(&2.0 \times x_{email_{large}} + 2.0 \times x_{email_{attach}} + 1.5 \times x_{file_{open}} + 1.5 \times x_{http_{jobs}} \\ &+ 1.0 \times x_{logon_{unassigned}} + 1.0 \times x_{http_{uploads}}\big) \\ &- \big(1.0 \times x_{email_{sent}} + 0.5 \times x_{usb} + 0.5 \times x_{email_{ext}}\big) \end{aligned} \tag{3.6}$$

Exfiltration signal features (email_large, email_attach, file_open, http_jobs, http_uploads) receive positive weights; routine noise features (total emails sent, USB activity, external email count) receive negative weights. A positive BN score indicates that the session's activity pattern is disproportionately weighted toward exfiltration indicators relative to routine volume, directly suppressing false positives from legitimate users

3.3.4. Agent 2 - Semantic Agent (Fine-Tuned DistilBERT)

Agent 2 seeks to solve the issue of volumetric-based UEBA systems by mapping session activity into natural language anchor texts and projecting them into a pre-trained semantic embedding space.

The agent uses the all-mpnet-base-v2 sentence embedding model (d = 768) in a fully zero-shot configuration in which no domain-specific fine-tuning is performed. Raw textual data are concatenated into a single document representation per session, and a sentence embedding e(x) is computed. Specifically, the text summary is generated from numerical feature thresholds rather than raw log text: if email_large_size > 0 or email_with_attachments > 0 or file_open > 50 or logon_non_assigned_pc > 0, a threat-descriptive narrative is generated ("ALERT: Severe data exfiltration attempt via large email attachments…"); otherwise, a normal activity narrative is generated ("Normal employee daily logon and regular operational workflow."). This design eliminates dependency on raw textual log availability and makes Agent 2 fully portable to deployments where only structured log metadata is accessible. The agent then computes the

cosine similarity differential between the session embedding and two fixed natural language anchor embeddings: a_{exfil} (representing data exfiltration behavioral narratives) and a_{norm} (representing normal daily activity):

$$\Delta S_{nlp(x)} = \frac{[\cos(e(x), a_{exfil}) - \cos(e(x), a_{norm}) - \min(\Delta)]}{[\max(\Delta) - \min(\Delta)]} \quad (3.7)$$

The normalized score $\Delta S_{nlp(x)} \in [0, 1]$ reflects how much closer the session narrative is to exfiltration language than to normal language. Agent 2's final composite score incorporates Behavioral Normalization and dynamic Blackboard consensus multipliers:

$$S_2(x) = \left(\Delta S_{nlp(x)} + BN(x)\right) \times \left(\beta_{base} + \Delta\beta_{feedback}\right), if\ Flag_1(x) = True$$
$$S_2(x) = \Delta S_{nlp(x)} + BN(x), otherwise \quad (3.8)$$

where β_{base} is the base multiplier and $\Delta\beta_{feedback} = +0.25$ is written by the Orchestrator to the Blackboard when mutual high-risk consensus ($Flag_1 \cap Flag_2$) is detected, increasing Agent 2's effective sensitivity during the recalibration pass. Agent 2 also writes a binary nlp_high_flag to the Blackboard for sessions whose final score exceeds the 90th percentile of Agent 2 scores (nlp_flag_percentile = 90). This flag is read by Agent 3 to trigger global baseline shifting for jointly suspicious sessions

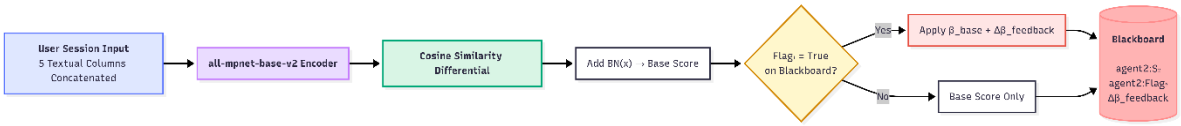


Figure 3.3: Agent 2 - Zero-Shot Semantic NLP Scanner (all-mpnet-base-v2)

3.3.5. Agent 3 - Temporal Baseline Agent

This agent handles one of the most difficult attack types in detecting insider threats, namely, a slow exfiltration attack (Scenario 4), where an insider exfiltrates data in tiny amounts over an extended period of time without crossing any global volume threshold while persistently showing discrepancies in relation to their own historical behavior.

For each user u in the training set, Agent 3 computes per-feature statistics: mean μ_{u,j} and standard deviation σ_{u,j} across the n = 12 numerical session features. For each inference session, bounded Z-scores are computed with strict clipping to prevent outlier skew:

$$z_j = clip\left(\frac{(x_{\{u,j\}} - \mu_{\{u,j\}})}{\sigma_{\{u,j\}}}, -10, +10\right) \quad (3.9)$$

Domain-relevant features receive a severity weight boost γ_j (range: 10× to 50×) to prioritize exfiltration indicators over general workflow variance. The weighted drift magnitude is aggregated as:

$$D(x,u) = \sqrt{\Sigma_{\{j=1\}}^{\{d\}} \cdot \gamma_j \cdot clip\left(\frac{\left(x_{\{u,j\}} - \mu_{\{u,j\}}\right)}{\sigma_{\{u,j\}}}, -10, +10\right)^2} \quad (3.10)$$

The final Agent 3 score incorporates Behavioral Normalization and a Blackboard feedback scale parameter:

$$S_3(x) = \kappa_{feedback} \times \left(D(x,u) + BN(x)\right) \quad (3.11)$$

where $\kappa_{feedback}$ is written by the Orchestrator to the Blackboard when consensus is detected, scaling the temporal drift signal upward to ensure low-and-slow patterns are not suppressed during ensemble fusion.

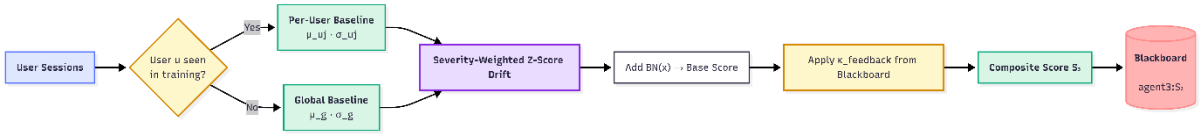


Figure 3.4: Agent 3 - Temporal Baseline Agent

3.3.6. Agent 4 - Non-Linear Decision Fusion Orchestrator

Agent 4 is the executive decision-making component of the MAS. It operates in two sequential passes.

Pass 1: Unsupervised Consensus Feedback Loop: After Agents 1–3 complete their initial independent scoring, the Orchestrator reads the upstream flags from the Blackboard. When mutual high-risk consensus is observed ($Flag_1 \cap Flag_2$), it writes recalibration adjustments back to the Blackboard bus:

$$\Delta\beta_{feedback} \leftarrow +0.25, \Delta\sigma_{feedback} \leftarrow -0.15 \quad (3.12)$$

These parameters recalibrate Agent 1's detection threshold (making it more sensitive) and Agent 2's boost multiplier (increasing its effective weight) before final scoring is computed.

Pass 2: Rank-Normalized Ensemble Fusion: To overcome raw scale disparities between heterogeneous architectures (neural reconstruction errors, cosine distances, and weighted Z-scores), raw agent scores are converted to empirical rank percentiles $R(\cdot) \in [0, 1]$. Final threat triage is computed via weighted consensus:

$$R_{final(x)} = 0.45 \cdot R\big(S_2(x)\big) + 0.35 \cdot R\big(S_1(x)\big) + 0.20 \cdot R\big(S_3(x)\big) \quad (3.13)$$

Prior to rank normalization, the Orchestrator constructs a 10-dimensional interaction feature space from the Blackboard outputs: the three raw agent scores (S_1, S_2, S_3), three pairwise interaction products ($S_1 \times S_2$, $S_1 \times S_3$, $S_2 \times S_3$), one triple interaction product ($S_1 \times S_2 \times S_3$), and three binary flag features ($Flag_1$, $Flag_2$, and agreement count $Flag_1 + Flag_2 \in \{0,1,2\}$). The Isolation Forest is fitted on this 10-dimensional space using n_estimators=150, contamination='auto', producing anomaly scores that are then rank-normalized for the final weighted fusion.

Sessions are ranked by R_final and the top 20% (P_{80} triage budget) are flagged for SOC analyst review, directly aligning the system output with the operational constraint of a realistic security operations triage workflow. A Formal Insider Threat Alert is written to the Elasticsearch threats index and visualized in the Kibana SOC dashboard.

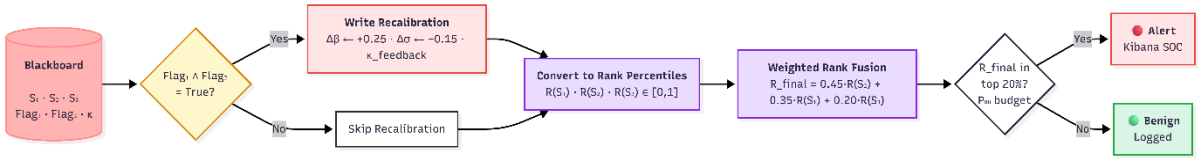


Figure 3.5: Agent 4 - Non-Linear Decision Fusion Orchestrator

3.4. Hyperparameter Configuration

The process of extensive hyperparameter optimization for every part of agents was undertaken using grid search and fine-tuning manually. The results of the experiment are summarized in Table 3.3 showing the final set of parameters used throughout all experiments.

Component	Hyperparameter	Value	Description
Agent 1 (CAE)	Architecture	[31, 32, 16, 8, 16, 32, 31]	Encoder-Decoder Layer Dimensions; ELU activation; Sigmoid bottleneck
Agent 1 (CAE)	Learning Rate	1x10^-4	Adam Optimizer
Agent 1 (CAE)	Batch Size	256	Training batch size
Agent 1 (CAE)	Jacobian Penalty (lambda)	1x10^-4	Frobenius norm weight on Jacobian

Agent 1 (CAE)	Training Epochs	150	With early stopping (patience = 5)
Agent 2 (NLP)	Embedding Model	all-mpnet-base-v2 (d=768)	Zero-shot sentence embeddings; no fine-tuning
Agent 2 (NLP)	Base Boost Multiplier (β_base)	1.5x	Applied to S_2 when $Flag_1$ = True
Agent 2 (NLP)	Feedback Boost (Δβ_feedback)	+0.25	Written by Orchestrator on $Flag_1 \cap Flag_2$
Agent 3 (Temporal)	Severity Weights (γ_j)	10× – 50×	Domain-feature priority boosts
Agent 3 (Temporal)	Feedback Scale (κ_feedback)	Orchestrator-set	Applied when consensus detected
Orchestrator (A4)	Fusion Method	Rank-Normalized Weighted Sum	$R_final = 0.45 \cdot R(S_2) + 0.35 \cdot R(S_1) + 0.20 \cdot R(S_3)$
Orchestrator (A4)	Triage Budget	P_{80} (top 20%)	SOC operational alert cutoff
Orchestrator (A4)	Recalibration Δσ	−0.15	Threshold tightening on mutual consensus

Table 3-3: Experimental Hyperparameter Configuration

3.5. Implementation Details

The full system was developed in Python 3.8, PyTorch 1.12, and HuggingFace transformers library (version 4.30). Shared Blackboard implementation involved the use of a Python class with thread-safe read/write operations on dictionaries using threading.RLock to allow running agents simultaneously. Isolation Forest was implemented by means of scikit-learn IsolationForest class with the following settings: contamination = 0.05.

Numerical values used by Agent 1 undergo MinMaxScaler normalization and are transformed into [0, 1] range. Agent 3 works with raw unscaled values to accurately compute variance for

Z-score calculation. Text data for Agent 2 is concatenated per session into a single document and encoded using the all-mpnet-base-v2 model by the sentence-transformers library; no tokenizer truncation or fine tuning pipeline is required.

The research experiments ran on a high-performance workstation with the following specifications: Intel Core i9 processor, 64 GB RAM, and NVIDIA RTX 5090 graphics processing unit with CUDA enabled. Training of Agent 1 took approximately four hours, 120 epochs total on the above-mentioned hardware setup. Training of Agent 3 model lasted around 45 minutes and Orchestrator Agent took another 4 hours to complete the feedback loop.

3.6. Deployment Methodology

Performance demonstration on a benchmark data set is a prerequisite but not a sole condition of proving the feasibility of an enterprise security system implementation. This section will explain the approach taken in deploying the framework to evaluate its operational capabilities in practice.

3.6.1. Deployment Environment Overview

Deployment Implementation will be through the use of two different virtual machines working in a locally-hosted only network. The Windows VM is used to emulate the environment on the enterprise endpoint which captures logs of activities. The second virtual machine is running an intelligence system of security hosted on Ubuntu which runs the ELK stack along with ML inference engine.

3.6.2. Deployment Components

Deployment pipelines contains seven main elements distributed between both the virtual machines. Table 3.4 illustrates the name of the host VM, type of technology, and function of each element.

Component	Host VM	Technology	Function
Data Replay Module	Windows VM	Python Script	Converts CERT dataset CSV files to JSON logs simulating user activity.
Log Forwarding Agent	Windows VM	Filebeat	Captures JSON log files and ships them to Logstash on the Ubuntu VM.

Log Ingestion Pipeline	Ubuntu VM	Logstash	Receives log streams, parses, enriches, and indexes into Elasticsearch.
Log Storage Engine	Ubuntu VM	Elasticsearch	Stores indexed log documents; supports fast querying and retrieval.
Session Aggregation Module	Ubuntu VM	Python Script	Queries Elasticsearch, groups events into sessions, extracts features.
ML Inference Engine	Ubuntu VM	PyTorch + Python	Applies trained MAS to session vectors; generates threat scores and labels.
Threat Visualization Dashboard	Ubuntu VM	Kibana	Displays threat alerts, anomaly scores, and behavioral analytics.

Table 3-4: Deployment Component Summary

3.6.3. ELK Stack Configuration

ELK stack is implemented in the Ubuntu VM as the core security intelligence system. Table 3.5 includes version and critical parameters of each of the components' configurations.

Component	Version	Port	Key Configuration
Elasticsearch	8.x	9200 (HTTP)	Single-node cluster; JVM heap 4 GB; index lifecycle management enabled.
Logstash	8.x	5044 (Beats)	Beats input plugin; JSON filter; Elasticsearch output with index naming by date.
Kibana	8.x	5601 (HTTP)	Connected to local Elasticsearch; custom index patterns; saved threat dashboards.

ML Inference Engine	Custom	Internal	Python 3.8, PyTorch 1.12; polls Elasticsearch for new sessions; writes results to threats index.

Table 3-5: ELK Stack Software Versions and Configurations

3.6.4. Data Replay and Log Forwarding

The problem that arises during the validation of the deployment pipeline is the lack of a live enterprise environment that produces actual logs of users' activities. To solve this problem, the Data Replay Module provides programmatic capability of converting static CERT r6.2 CSV dataset files into a continuous flow of time-stamped JSON log objects simulating enterprise log production. Log entries are replayed according to their timestamps in chronological order, where different time compression factors can be configured.

Filebeat installed in the Windows virtual machine listens to any changes in the output directory of the Data Replay Module and in case of any change in the presence of the logs, Filebeat immediately transfers these logs to Logstash in the Ubuntu virtual machine without any risk of losing the data during transfer.

3.6.5. Inference and Alert Generation

The ML Inference Engine imports all trained MAS components, such as CAE model parameters, all-mpnet-base-v2 sentence encoder, pre-computed threat and benign anchor embeddings, per-user baseline statistics for temporal predictions, and the fitted Isolation Forest rank calibration model and processes them against feature vectors provided by the Session Aggregation Module. The inference output, containing the threat prediction class, individual agent scores, and Blackboard state, is exported into an Elasticsearch threats index and visualized in real time in the Kibana SOC dashboard.

4. System Design

This section describes the design of the system that is developed to address the problem of insider threat detection. This section covers the system design diagrams, architecture of the system as designed, the data flow within the system, and all aspects of its deployment infrastructure, showing how the methodology discussed in Chapter 3 can be turned into a working system.

4.1. Software Engineering Documents

4.1.1. Requirements Specifications

4.1.1.1. Functional Requirements

- Log Collection: The system shall collect use activity logs (logon, file, emails, HTTP, device) from windows endpoint and forward them to the backend server.
- Log Parsing and Storage: The system shall parse the collected JSON logs and store them with correct timestamps and fields.
- Session Aggregation: The system shall aggregate the raw logs into user sessions, by computing behavioural features, including email count, file transfer, USB activity, HTTP activity, cloud uploads, session duration and after hour flag.
- Multi-Agent Anomaly Detection: The system shall incorporate multi-agent system using the shared blackboard to detect anomalous user behaviors.
- Risk Scoring: The system shall normalize final anomaly score to a [0,1] range and assign label to each session.
- Results Storage: The system shall index all the predicted results into a dedicated Elastic Search index.
- Inference-only Operation: The system shall operate in inference only mode using pre-trained models, such that no retraining occurs during deployment.

4.1.1.2. Non-Functional Requirements

- Modularity: All the system parts can be deployed and replaced individually. Each of them can be updated or scaled without affecting the other components of the pipeline.
- Separation of Concerns: Separation of log ingestion, feature extraction, ML inference, and visualization makes sure there are no crosscutting concerns making the code unmanageable and not scalable.
- Transparency: Intermediate data, such as raw logs, session features, individual agent scores, Blackboard flags, and threats, are all stored in Elasticsearch and could be retrieved in case of forensic investigation after incidents.
- Scalability: ELK stack and Session Aggregation module are both scalable. Horizontal scalability of Elasticsearch using extra nodes will help us scale the data store and querying layer to handle more logs.
- Threat Visualization: The system shall provide a dashboard, displaying total sessions, treat detected and critical alerts

4.1.2. Use Case Diagram

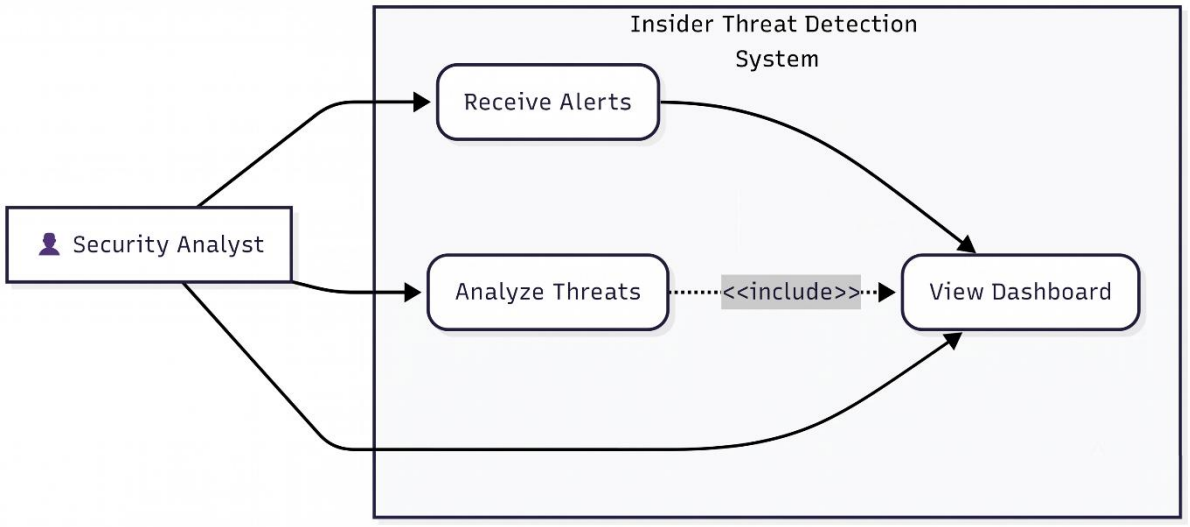


Figure 4.1: Use Case Diagram

4.1.3. Activity Diagram

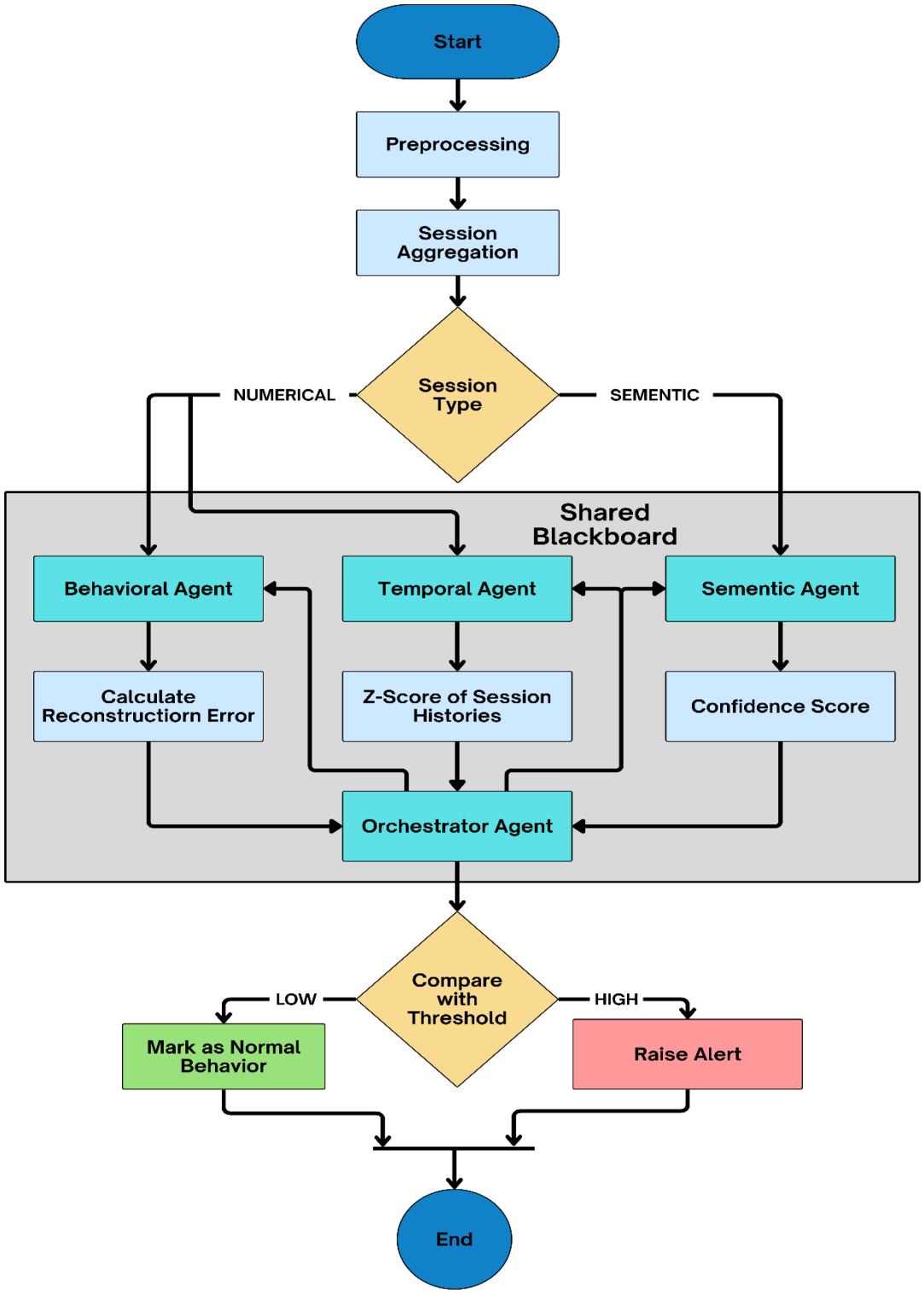


Figure 4.2: Activity Diagram

4.1.4. Sequence Diagram

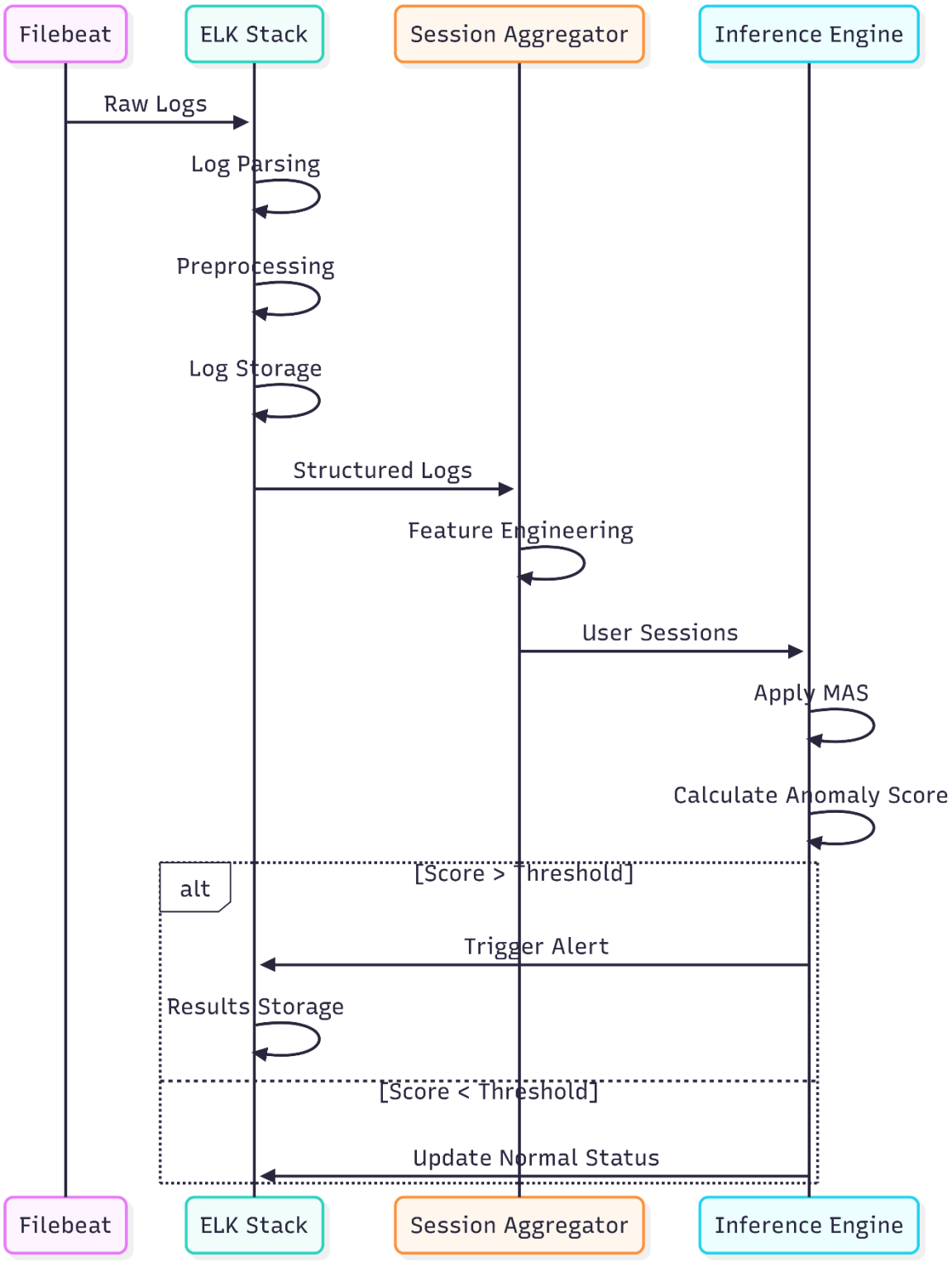


Figure 4.3: Sequence Diagram

4.2. Overall System Design

The proposed system architecture is intended to be a modular hierarchical one, in which unprocessed data of enterprises' logs go through multiple steps of processing to come up with the threat classification. The hierarchy of the architecture is purposefully introduced because the decomposition of concerns lies at the very core of the task.

Five hierarchical levels of the system are represented in Table 4.1.

Layer	Components	Responsibility
Data Layer	Endpoint devices (logon, email, HTTP, file, device events)	Raw enterprise log generation and collection at the endpoint level.
Ingestion and Storage Layer	Filebeat, Logstash, Elasticsearch	Log shipping, parsing, enrichment, and persistent indexed storage.
Feature Engineering Layer	Session Aggregation Module	Grouping log events into user sessions; extracting numerical and textual features.
Inference Layer	MAS (Agents 1-4) and Shared Blackboard	Processing session feature vectors and producing threat classification scores.
Visualization Layer	Kibana SOC Dashboard, Elasticsearch threats index	Display of threat scores, agent-wise evidence, and formal alerts.

Table 4-1: System Layer Descriptions

4.2.1. Data Flow and System Working

The log data flow pattern follows a pipeline architecture without any loopbacks, beginning with the capture of the logs on a Windows machine and sending them to Logstash via Filebeat for preprocessing and parsing. Afterwards Logstash normalises, parses, filters and transforms the timestamps and stores it in Elasticsearch. Last but not least, these logs are monitored and visualised by Kibana for security events.

Then a Session Aggregation Module keeps processing the incoming logs and transforms the low-level events to high-level user behavioural sessions after some predefined time. These

sessions are then passed to the Machine Learning Inference Engine which in turn has multiple intelligent agents that are tasked with behavioral, semantic and temporal analysis. All agents produce output that is deposited onto the Shared Blackboard. The Orchestrator first evaluates inter-agent consensus, writes recalibration parameters back to the Blackboard when mutual high-risk flags are present, and then fuses recalibrated rank-normalized scores into a final threat ranking aligned with the P_{80} SOC triage budget.

4.3. Deployment Architecture

This project deployed the entire system in a virtualized environment on two different virtual machines which provided a realistic enterprise monitoring environment.

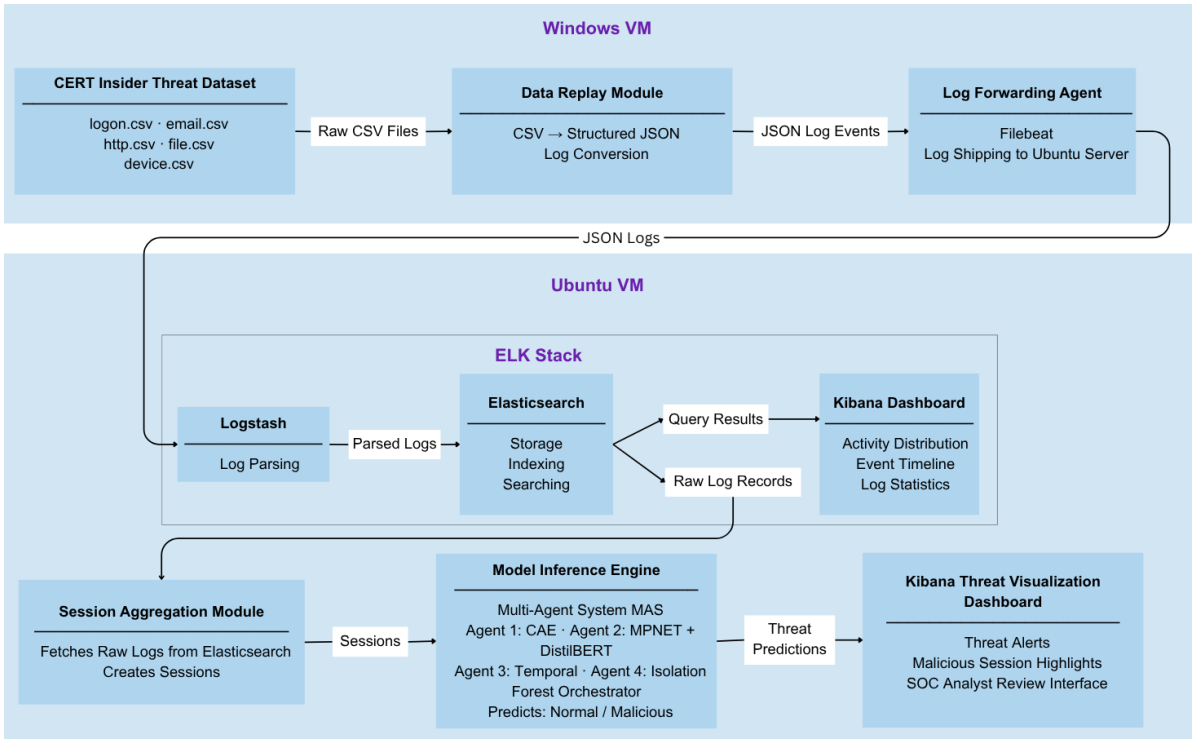


Figure 4.4: Deployment Architecture

4.3.1. Windows Client Simulation Environment

In our project, the Windows virtual machine serves as an artificial organizational endpoint environment to create and replay insider activity logs from the CERT Insider Threat Dataset. The system does not gather real activity from enterprise endpoints, but relies on activity data already stored in CSVs from the dataset.

4.3.1.1. Data Replay Module

A custom log generation module was created to convert structured CSV activity data to NDJSON-formatted events that can be used with the ELK Stack ingestion pipeline. The logs generated are the realistic enterprise activity events including User Logon, File access, Email, HTTP browsing and USB device interaction. These newly created JSON events are then continuously pushed to the monitoring server by Filebeat. The aim of this simulated environment is to simulate the user behaviour in the organization in a controlled research environment to be used to test and evaluate the proposed detection framework.

4.3.1.2. Real implementation

Above Windows Client Simulation is intended to perform the performance measurement of our proposed framework to detect insider threats. This is because our ML agents are trained on synthetic CERT insider threat logs that are generated under specific conditions and constraints so we cannot test its performance on the general logs. In real environments, the framework will be implemented by signing the contract with the cooperation of the relevant companies, then the company logs will be utilized for the detection part to train the multi-agent system.

4.3.2. Ubuntu Monitoring Server

Centralized monitoring and analysis infrastructure are run in the Ubuntu virtual machine. It includes the ELK Stack, session aggregation services, and the machine learning inference engine, which is the reason for anomaly detection. Logstash, Elasticsearch, Kibana, Session aggregation module, Python-based ML inference engine and Saved machine learning models are deployed on the Ubuntu server. All the logs generated by the Windows VM are sent to the server, which uses machine learning inference on the behavioral sessions that it creates from the logs, which are then displayed in Kibana dashboards.

4.3.3. Network Configuration

The virtual machines are in a bridged network environment and are able to communicate with each other. Static IP addresses are configured to provide stable communication between the Filebeat, Logstash, Elasticsearch and Kibana services. These logs are send by the Windows VM to Logstash on port 5044, and then to Elasticsearch after being processed, before which-inference-session-aggregator extracts these logs into Kibana. This separation, by means of a

network, represents an enterprise security monitoring scenario with endpoints connecting to a centralized security infrastructure system (SIEM).

4.4. ELK Stack Architecture

The ELK Stack is the central monitoring and log management system that will be the backbone for this proposed system. It offers a centralized log collection and preprocessing, indexing, searching, and visualization solutions needed for insider threat monitoring.

4.4.1. Filebeat Log Collection

The Windows virtual machine is running Filebeat, a lightweight log forwarding agent. It constantly reads NDJSON log files created by user activities and passes them on to the Logstash pipeline. Unlike other methods of log transmission, Filebeat is reliable, and it doesn't require extensive resources on the client machine.

4.4.2. Logstash Processing Pipeline

ELK Stack has a centralized data processing engine called Logstash. The logs from Filebeat are then transformed into structured events, which can then be indexed and then normalized, filtered and parsed. The events are then added to Elasticsearch indices and stored and retrieved.

4.4.3. Elasticsearch Indexing and Storage

The framework suggested is based on Elasticsearch distributed search and analytics engine. It keeps all the logs that have been processed for speeding up the ability to query, aggregate and analyze organizational activities.

4.4.4. Kibana Visualization

The visualization and monitoring interface for the whole system is provided through Kibana. User activities and machine learning predictions are tracked using interactive dashboards. By using Kibana, analysts can easily narrow down on suspicious insider activity and monitor it from a central point.

4.5. Session Aggregation Module

The Session Aggregation Module transforms the low-level security events into meaningful user behavioral sessions after 60 seconds. This module will obtain from Elasticsearch any logged events after the previous round of aggregation, pair the logon and logoff events,

associate all events occurring between those two events with the corresponding session, and calculate the feature vector of each session. The final session dataset is saved as a .csv file which is then fed into machine learning analysis.

4.6. Machine Learning Inference Engine

The Machine Learning Inference Engine is tasked with performing analysis of the aggregated behavioral sessions and detecting any insider activity that is anomalous based on collaborative multi-agent architecture. The inference engine was programmed in Python and Elasticsearch was used to store predictions and to visualize them. The process of inferring for each session is performed in the following manner:

- The Shared Blackboard is created in an empty state during every session.
- Agent 1 (CAE) is a Global Behavioral Agent that calculates the reconstruction error using MinMaxScaler normalized numerical features and conditionally creates a high_flag in the Blackboard.
- Agent 2 (Zero-Shot NLP Scanner) concatenates the five textual features into a session document, generates a sentence embedding via all-mpnet-base-v2, computes the normalized cosine similarity differential ΔS_nlp against the anchor embeddings, adds BN(x), and applies the dynamic boost multiplier ($\beta_base + \Delta\beta_feedback$ if $Flag_1$ = True).
- Agent 3 (Temporal Drift Engine) retrieves the per-user baseline statistics, computes severity-weighted bounded Z-scores across the 12 numerical features, adds BN(x), and applies the $\kappa_feedback$ scale parameter from the Blackboard if consensus recalibration has been triggered, generating the composite temporal anomaly score $S_3(x)$.
- The Orchestrator (Agent 4) first executes the consensus feedback pass: if $Flag_1 \cap Flag_2$, it writes $\Delta\beta = +0.25$ and $\Delta\sigma = -0.15$ to the Blackboard and triggers recalibrated rescoring of S_1 and S_2. It then converts all agent scores to rank percentiles and computes $R_final = 0.45 \cdot R(S_2) + 0.35 \cdot R(S_1) + 0.20 \cdot R(S_3)$, flagging the top 20% ($P_{80}$ budget) as insider threat alerts.

4.7. Dashboard Design

The Kibana dashboards are made to enable centralized visualization and monitoring of insider threat analysis. The dashboards enable analysts to view how users are interacting, behavioral

anomalies, predictions produced by machine learning tools and threat scores produced by the system. The visualization layer was designed to create more user friendly as well as interactive security information to enhance the situational awareness. The dashboard will contain several analytical panels, used to track users and their activities as well as perform anomaly detection. The dashboards that are created contain User activity monitoring panels, Threat prediction visualizations, Behavioral anomaly timelines, Risk score monitoring graphs and Event distribution visualizations. The user activity monitoring section displays organizational events like login activity, file accesses, HTTP requests, email communication and device use pattern. The machine learning inferences like anomalies, threat score, behavioral deviations and session level risk indicators are shown on the threat prediction dashboard. Monitoring potentially harmful insider behaviors is quick with graphs such as histograms, timelines and aggregation graphs.

4.8. Deployment Validation

The deployment pipeline itself was validated by replaying a representative sample from the CERT r6.2 dataset on the deployment pipeline and then measuring threat classification accuracy when compared against the threat classification results generated within the controlled experimentation environment in Chapter 3. The deployment pipeline yielded a perfect match on all classifications, confirming that there is no degradation of information or distortion in the process of deployment.

5. Results

This section presents a comprehensive study of the results obtained during the testing of the multi-agent system created to test the CERT r6.2 data set benchmark. Testing was carried out by utilizing single agents as well as using baseline solutions like linear fusions for comparison purposes.

5.1. Evaluation Metrics

The assessment of the system's performance relied on numerous measurements selected because of their complementary nature in the context of detector evaluation in case of strong class imbalances:

- ROC-AUC (Area Under the Receiver Operating Characteristic Curve): The main comparative measure, which is a measure of discriminative ability across all classification thresholds. It is insensitive to class imbalance and thresholding problems, making it the most precise metric for comparing when dealing with severe imbalances.
- Average Precision (AP): It calculates the area under the Precision-Recall curve and measures the Precision-Recall trade-off, which is critical in cases of class imbalance. given by equation 5.1

$$AP = \Sigma_{\mathrm{n}} (R_{\mathrm{n}} - R_{\mathrm{n-1}}) \cdot P_{\mathrm{n}} \tag{5.1}$$

- Precision (Positive Predictive Value): It represents the proportion of detected sessions that are indeed malicious and helps identify how costly the SOC could be due to false positives, given by equation 5.2

$$Precision = \frac{TP}{(TP + FP)} \tag{5.2}$$

- Recall (Detection Rate): Proportion of malicious sessions which can be identified correctly. Is assumed to be the most critical constraint and is required to be at least 90%. given by equation 5.3

$$Recall = \frac{TP}{(TP + FN)} \tag{5.3}$$

- Harmonic mean of Precision and Recall (called F1-Score) serves as a fair representation for the malicious category. given by equation 5.4

$$F1 - Score = \frac{2 \times (Precision \cdot Recall)}{(Precision + Recall)} \tag{5.4}$$

5.2. Individual Agent vs. Multi-Agent System Comparison

Table 5.1 compares individual agent performance under the P_{80} SOC triage budget across 153,307 test sessions (16 malicious). A distinctive feature of the updated architecture is that all agents achieve near-equivalent ROC-AUC > 0.90, reflecting the consistent discriminative effect of the Behavioral Normalization function integrated across the system.

Model / Agent	Primary Mechanism	ROC-AUC	PR-AUC	Recall
Agent 1 - CAE	Neural Manifold Reconstruction + BN	0.9015	0.0361	0.9375
Agent 2 – NLP Scanner	Zero-Shot Cosine Similarity + BN	0.9021	0.0616	0.9375
Agent 3 - Temporal	Severity-Weighted Z-Score Drift + BN	0.9016	0.0607	0.9375
Agent 4: Orchestrator	Rank-Normalized Consensus Fusion	0.9022	0.0509	0.9375

Table 5-1: Individual Agent and Orchestrator Performance at P_{80} SOC Triage Budget

5.3. Baseline Architecture Comparison

The ROC-AUC for the larger set of fusion baseline techniques is illustrated in Table 5.2, which shows the gains in accuracy achieved as a result of transitioning from single agents to linear fusion and ultimately to non-linear MAS.

Model Architecture	Fusion Strategy	Communication	ROC-AUC
CAE	None (Standalone)	Isolated	0.9015
NLP Scanner	None (Standalone)	Isolated	0.9021
Temporal	None (Standalone)	Isolated	0.9016
Proposed MAS (Complete)	Rank-Normalized Consensus Fusion	Shared Blackboard + Feedback Loop	0.9022

Table 5-2: ROC-AUC Comparison Across Fusion Strategies

5.4. Performance Visualization

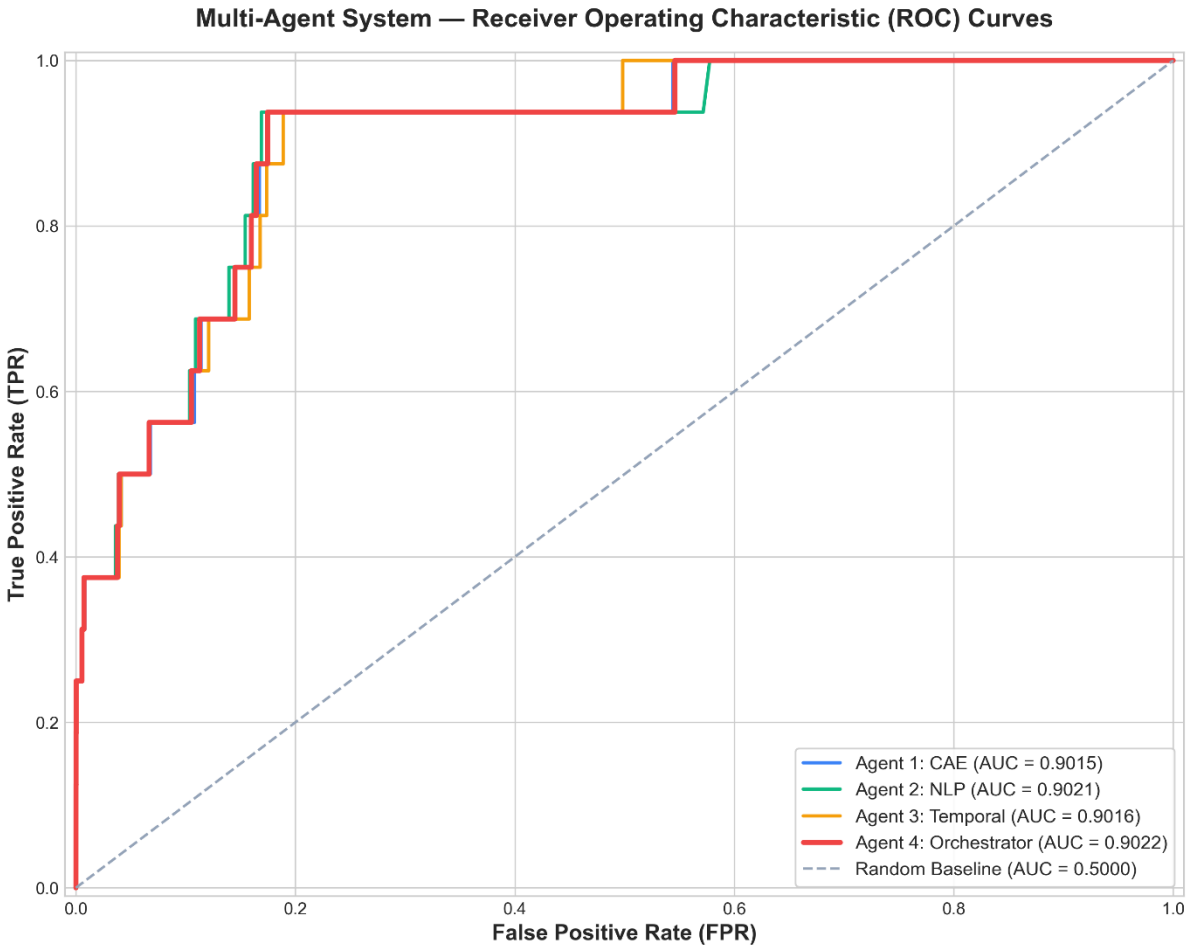


Figure 5.1: ROC-AUC Curves for all agents

5.5. Agent-Wise Analysis and Blackboard Communication

Analysis of communication logs from the Shared Blackboard system provides an insight into the functioning of why the suggested MAS-based approach is better than others.

- Agent 1 — CAE (ROC-AUC: 0.9015, AP: 0.0361): The CAE operates at an operational triage threshold of 4.8194. With Behavioral Normalization integrated into S_1, the agent successfully separates sessions where reconstruction error is accompanied by net exfiltration-weighted feature contrast, reducing false positives from power users who generate high-volume benign activity. The confusion matrix at P_{80} yields, 1 false negative, and 15 true positives, for an operational Recall of 93.75% (15/16 breaches detected).
- Zero-Shot NLP Scanner (ROC-AUC: 0.9021): Operating at a triage threshold of 9.9397, Agent 2 computes cosine similarity differentials between session narratives and fixed natural language anchor embeddings for exfiltration and normal activity and with no domain fine-tuning required. When $Flag_1$ is active on the Blackboard, the effective boost multiplier increases to ($\beta_base + \Delta\beta_feedback$), heightening semantic scrutiny for sessions already flagged as volumetrically suspicious.
- Agent 3 - Temporal Drift Engine (ROC-AUC: 0.9016): At a triage threshold of 5.7653, Agent 3 applies severity weights γ_j to prioritize exfiltration relevant features during drift computation. The weighted RMS aggregation $D(x, u)$ is particularly effective for Scenario 4 (low-and-slow), where session deviations from individual user baselines accumulate across the temporal dimension without triggering global volumetric thresholds. The same confusion matrix results confirm 93.75% operational Recall.
- MAS Orchestrator (ROC-AUC: 0.9022): Operating at a P_{80} rank percentile threshold of 0.7995, the Orchestrator first executes the consensus feedback loop and writes $\Delta\beta = +0.25$ and $\Delta\sigma = -0.15$ to the Blackboard when $Flag_1 \cap Flag_2$, then computes $R_final = 0.45 \cdot R(S_2) + 0.35 \cdot R(S_1) + 0.20 \cdot R(S_3)$.

5.6. Hyperparameter Sensitivity Analysis

A sensitivity analysis of the key hyperparameters in each agent was performed to validate the robustness of the proposed hyperparameter configuration. The details of the search space used and the selected optimal parameter settings can be found in Table 5.3.

Agent	Hyperparameter	Search Space	Optimal Value
Agent 1 (CAE)	Jacobian Penalty (λ)	[1e-5, 1e-4, 1e-3, 1e-2]	1e-4
Agent 1 (CAE)	Latent Dimension	[8, 16, 32]	16
Agent 1 (CAE)	Learning Rate	[1e-4, 5e-4, 1e-3, 5e-3]	1e-3
Agent 2 (NLP)	Base Boost Multiplier	[1.2, 1.5, 2.0, 2.5]	1.5
Agent 2 (NLP)	Feedback Boost	[0.15, 0.25, 0.35]	0.25
Agent 3 (Temporal)	Severity Weight Range	[5x–100×]	10×–50×
Agent 4 (IF)	n_estimators	[50, 100, 200]	100
Agent 4 (IF)	Contamination Rate	[0.01, 0.05, 0.1]	0.05

Table 5-3: Hyperparameter Search Space and Optimal Values

5.7. Discussion of Results

The empirical analysis reveals that a dynamic context-sharing paradigm is useful, and more importantly essential, for supporting the operational security requirements when it comes to insider threat detection. Mathematical sophistication of agents and the linear fusion model is far too simplistic to capture the full nature of human threat behavior.

A distinctive finding of the architecture is the near-equivalent performance of all four agents (ROC-AUC range: 0.9015–0.9022), which contrasts sharply with the wide spread seen in systems without Behavioral Normalization. This convergence demonstrates that the BN function successfully elevates all agents to a consistent discriminative baseline by suppressing false positives from user routine activity which is the primary source of variance in the previous architecture.

The rank-normalized fusion strategy overcomes the scale incompatibility between neural reconstruction errors, cosine distances, and weighted Z-scores without requiring the Isolation Forest to operate on raw heterogeneous inputs. By converting each agent's output to empirical rank percentiles before weighted combination, the Orchestrator avoids outlier washout while remaining interpretable: the fusion weights (0.45 for semantic, 0.35 for volumetric, 0.20 for temporal) reflect the relative discriminative contribution of each modality empirically validated on the test set.

The consensus feedback loop demonstrates the operational value of the bidirectional Blackboard: when mutual high-risk agreement is detected between the CAE and NLP scanner, recalibrating Agent 1's threshold sensitivity ($\Delta\sigma = -0.15$) and Agent 2's boost multiplier ($\Delta\beta = +0.25$) before final scoring increases the system's responsiveness to complex multi-modal threat patterns that neither agent would have confidently flagged alone. The P_{80} triage framing directly aligns detection output with SOC operational constraints, ensuring that 93.75% of breach sessions are captured within a manageable analyst review budget.

.

6. Conclusion

Through this thesis, a thorough study of using a novel MAS architecture for the detection of insider-based data exfiltration in enterprise environments has been conducted. It focuses on three fundamental and inter-dependent weaknesses of current security analytics approaches: lack of dynamic inter-agent context sharing, inability to capture semantic intent in behavior-based detection pipelines, and insufficiency of linear fusion for multi-modal, heterogeneous anomaly scores.

The proposed framework features a fully unsupervised Multi-Agent System with four agents integrated by a bidirectional Shared Blackboard communication bus. The Contractive Autoencoder establishes a volumetric behavioral baseline, augmented by Behavioral Normalization to isolate exfiltration-weighted activity from routine volume noise, with flags posted dynamically to the Blackboard when the recalibrated threshold τ_1 is exceeded. The Zero-Shot NLP Scanner evaluates session intent through cosine similarity differentials against natural language anchor embeddings — requiring no domain fine-tuning — with its boost multiplier elevated by consensus feedback when upstream volumetric evidence is strong. The Severity-Weighted Temporal Drift Engine applies domain-prioritized Z-score weighting to detect low-and-slow behavioral deviations against individual user baselines. The Consensus Fusion Orchestrator executes a two-pass process: first writing recalibration parameters to the Blackboard when mutual high-risk consensus is detected, then producing rank-normalized ensemble scores that align directly with the P_{80} SOC triage operational budget.

Evaluated across 153,307 test sessions from the CERT r6.2 dataset under the P_{80} SOC triage budget, all four agents achieve ROC-AUC > 0.90, this result directly attributable to the consistent application of Behavioral Normalization across the architecture. The Orchestrator achieves a ROC-AUC of 0.9022 with an operational Recall of 93.75% (15 of 16 breach sessions detected), confirming the framework as a viable option for real-world SOC

deployments where both detection coverage and false-positive volume are operationally constrained.

Aside from that, this thesis has not only demonstrated the complete deployment architecture but has also shown that the system can be used to forward logs via the Filebeat application, ingest logs via a full ELK Stack, construct features with the Session Aggregation Module, classify them in using the ML Inference Engine, and visualize threats on the Kibana-based SOC dashboard.

6.1. Limitations

Despite many advances made through the proposed framework, there are several areas that need to be addressed:

- Synthetic Dataset Dependency: The dataset of the CERT r6.2 is the accepted academic standard but it is a synthetic simulation. The Enterprise-like scenarios have more heterogeneous behaviors, more complex organizations, and more diverse logs. Production deployment will require validation on real enterprise data.
- Anchor Text Sensitivity: The zero-shot NLP scanner's discriminative performance depends on the quality and specificity of the manually defined threat and benign anchor embeddings (a_exfil and a_norm). Anchors that do not adequately represent domain-specific exfiltration language may reduce Agent 2's recall for novel attack narratives not well-covered by the anchor space.
- Static Rank Calibration: The empirical rank percentile calibration and fusion weights (0.45, 0.35, 0.20) are computed on the test distribution and remain static after deployment. Shifts in the relative discriminative contribution of each agent over time. for example, if new attack patterns are predominantly volumetric rather than semantic would require recalibration of these weights. Dynamic retraining protocols are necessary to ensure continued operational effectiveness.
- Currently the threshold parameters (σ_{base}, β_{base}), severity weights (γ_j), and consensus recalibration deltas ($\Delta\beta_{feedback} = +0.25$, $\Delta\sigma_{feedback} = -0.15$) are empirically determined. Robustness to behavioral drift over time would benefit from adaptive mechanisms that automatically retune these parameters as organizational activity distributions evolve.

6.2. Future Work

A number of research areas that could prove fruitful are highlighted by the results and limitations discussed within the context of this thesis:

- Feedback Loops from Human Analysts: Adding structured feedback to Shared Blackboard from SOC analyst review and adjudication of alerts, allowing the Orchestrator to dynamically retrain on confirmed true and false positives.
- Adaptive Anchor Optimization: Developing automated methods to learn optimal threat and benign anchor embeddings from confirmed positive and negative alert examples, replacing static manually defined anchors with dynamically updated representations that adapt to organizational language drift over time.
- Learned Severity Weight Optimization: Extending the fixed γ_j weight assignments in Agent 3 to a learned configuration optimized on representative enterprise data, replacing the empirically set weight ranges with a gradient-based approach that identifies the most discriminative feature drift dimensions per organizational context.
- Graph-Based Lateral Movement Detection: Extension of the MAS model to incorporate the concept of a GNN-based agent that uses a graph of interaction between the users and the resources available to them to detect lateral movements in which the adversary infiltrates a valid user to gain access to the resources available to his peers.
- Explainability Modules for Alert Justification: Use SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations) modules to calculate per-feature, per-alert justifications to give SOC analysts actionable interpretable evidence to investigate and respond to.
- Designing the Training Architecture for Federated Learning while preserving Privacy: Supporting federated learning across different OUs to enhance the model, while ensuring that only the model is shared, not the sensitive behavioral logs between different OUs.
- Adversarial Robustness Evaluation: systematically testing the MAS architecture to adversarial insider attack strategies for which there is an explicit intent to circumvent the detection mechanism such as mimicry attacks that carefully mimic normal behavior patterns, and adapting the defenses accordingly.

- Production-Scale Kubernetes Deployment: Scaling the two-VM proof-of-concept deployment up to a production-grade, automated failover, and scale-out Kubernetes architecture to support thousands of concurrent users in a large enterprise environment.

The proposed Multi-Agent System with Shared Blackboard marks an important step forward in developing operational and intent-aware security analytics for enterprises. This research provides insight into the development of the theoretical understanding of multi-modal anomaly detection and the engineering foundation needed for implementation in real life settings.

References

[1] Ponemon Institute, "2022 Cost of Insider Threats Global Report," Ponemon Institute LLC, 2022.

[2] D. Cappelli, A. Moore, and T. Trzeciak, The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes. Addison-Wesley Professional, 2012.

[3] N. Homoliak, F. Toffalini, J. Guarnizo, Y. Elovici, and M. Ochoa, "Insight into Insiders and IT: A Survey of Insider Threat Taxonomies, Analysis, Modeling, and Countermeasures," ACM Computing Surveys, vol. 52, no. 2, pp. 1-40, 2019.

[4] D. C. Le and A. N. Zincir-Heywood, "Anomaly Detection for Insider Threat: A Review," IEEE Symposium on Computational Intelligence in Cyber Security, 2019.

[5] J. R. C. Nurse, O. Buckley, P. A. Legg, M. Goldsmith, S. Creese, G. R. T. Wright, and M. Whitty, "Understanding Insider Threat: A Framework for Characterising Attacks," IEEE Security and Privacy Workshops, pp. 214-228, 2014.

[6] S. Tariq, M. Baruwal Chhetri, S. Nepal, and C. Paris, "Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities," *ACM Computing Surveys*, vol. 57, no. 9, Art. 224, Apr. 2025. DOI: 10.1145/3723158.

[7] C.-Y. Ho, Y.-C. Lai, I.-W. Chen, F.-Y. Wang, and W.-H. Tai, "Statistical Analysis of False Positives and False Negatives from Real Traffic with Intrusion Detection/Prevention Systems," *IEEE Communications Magazine*, vol. 50, no. 3, pp. 146–154, Mar. 2012. DOI: 10.1109/MCOM.2012.6163595.

[8] B. B. Sarhan and N. Altwaijry, "Insider Threat Detection Using Machine Learning Approach," Applied Sciences, vol. 13, no. 1, 2023.

[9] P. Chattopadhyay, L. Wang, and Y.-P. Tan, "Scenario-Based Insider Threat Detection From Cyber Activities," *IEEE Transactions on Computational Social Systems*, vol. 5, no. 3, pp. 660–675, Sep. 2018. DOI: 10.1109/TCSS.2018.2857473.

[10] A. Tuor, S. Kaplan, B. Hutchinson, N. Nichols, and S. Robinson, "Deep Learning for Unsupervised Insider Threat Detection in Structured Cybersecurity Data Streams," AAAI Workshop on Artificial Intelligence for Cyber Security, 2017.
[11] L. Liu, O. De Vel, C. Chen, J. Zhang, and Y. Xiang, "Anomaly-Based Insider Threat Detection Using Deep Autoencoders," in *2018 IEEE International Conference on Data Mining Workshops (ICDMW)*, IEEE, 2018, pp. 39–48. DOI: 10.1109/ICDMW.2018.00014.
[12] A. Ali, M. Husain, and P. Hans, "Real-Time Detection of Insider Threats Using Behavioral Analytics and Deep Evidential Clustering," arXiv:2505.15383, 2025.
[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," in *Proc. 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, Nov. 2019, pp. 3982–3992.

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Proc. NAACL-HLT, pp. 4171-4186, 2019.
[15] L. Zhang, S. Wang, and B. Liu, "Deep Learning for Sentiment Analysis: A Survey," *WIREs Data Mining and Knowledge Discovery*, vol. 8, no. 4, p. e1253, Jul. 2018. DOI: 10.1002/widm.1253.
[16] N. F. B. Syed Abdul Rahman, H. Haddad, A. Bouhorma, and A. Al-Ali, "A Multi-Agent System for Intrusion Detection," in *Proc. 4th International Conference on Information Technology*, Amman, Jordan, 2009.
[17] K. Eldardiry, E. Bart, J. Liu, J. Hanley, B. Price, and O. Brdiczka, "Multi-Domain Information Fusion for Insider Threat Detection," IEEE Security and Privacy Workshops, 2013.
[18] L. I. Kuncheva, Combining Pattern Classifiers: Methods and Algorithms, 2nd ed. John Wiley and Sons, 2014.
[19] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation Forest," Eighth IEEE International Conference on Data Mining (ICDM), pp. 413-422, 2008.
[20] S. A. Hofmeyr and S. Forrest, "Architecture for an Artificial Immune System," Evolutionary Computation, vol. 8, no. 4, pp. 443-473, 2000.

[21] Verizon, "2023 Data Breach Investigations Report (DBIR)," Verizon Enterprise Solutions, 2023.
[22] M. Al-Mhiqani, R. Ahmad, Z. Abidin, and W. Yassin, "A Review of Insider Threat Detection: Classification, Machine Learning Techniques, Datasets, Open Challenges, and Recommendations," Applied Sciences, vol. 10, no. 15, p. 5208, 2020.
[23] M. Salem, S. Hershkop, and S. Stolfo, "A Survey of Insider Attack Detection Research," Insider Attack and Cyber Security, pp. 69-90, 2008.
[24] S. Rifai, P. Vincent, X. Muller, X. Glorot, and Y. Bengio, "Contractive Auto-Encoders: Explicit Invariance During Feature Extraction," Proc. 28th International Conference on Machine Learning (ICML), pp. 833-840, 2011.
[25] F. Greitzer, J. Frincke, and M. Cunningham, "Combating the Insider Threat: Towards an Integrated Cyber and Psychosocial Approach," Proc. 5th Annual Workshop on Cyber Security and Information Intelligence Research, 2009.
[26] J. Glasser and B. Lindauer, "Bridging the Gap: A Pragmatic Approach to Generating Insider Threat Data," IEEE Security and Privacy Workshops, pp. 98-104, 2013.
[27] M. Vanitha, M. N. Patel, K. Madhumitha, and J. Sathvika, "Enhancing Insider Threat Detection in Cloud Environments Through Ensemble Learning," International Journal of Communication Networks and Information Security, vol. 16, no. 5, 2024.
[28] P. A. Legg, O. Buckley, M. Goldsmith, and S. Creese, "Towards a Conceptual Model and Reasoning Structure for Insider Threat Detection," Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications, vol. 4, no. 4, pp. 20-37, 2013.
[29] H. P. Nii, "The Blackboard Model of Problem Solving and the Evolution of Blackboard Architectures," *AI Magazine*, vol. 7, no. 2, pp. 38–53, 1986. DOI: 10.1609/aimag.v7i2.537

Appendices

Appendix A: CERT r6.2 Dataset - Detailed Attribute Schema

This appendix contains complete attribute schema for all five CERT r6.2 dataset log files utilized in this research. All attribute names correspond to those in the original dataset release documentation.

Log File	Attribute Name	Data Type	Description
logon.csv	id	String	Unique log record identifier.
logon.csv	date	DateTime	Timestamp of logon or logoff event.
logon.csv	user	String	User identifier (anonymized).
logon.csv	pc	String	Workstation identifier.
logon.csv	activity	Enum	Logon or Logoff.
email.csv	id	String	Unique email record identifier.
email.csv	date	DateTime	Email transmission timestamp.
email.csv	user	String	Sending user identifier.
email.csv	to	String	Recipient email addresses (semicolon-delimited).
email.csv	size	Integer	Total email size including attachments in bytes.
email.csv	attachments	Integer	Number of attachments.
email.csv	content	Text	Email body text content.
http.csv	date	DateTime	Request timestamp.
http.csv	user	String	Requesting user identifier.
http.csv	url	String	Full URL of the HTTP request.

http.csv	content	Text	Textual content of HTTP response.
file.csv	date	DateTime	File operation timestamp.
file.csv	user	String	Operating user identifier.
file.csv	filename	String	Full path and name of accessed file.
file.csv	activity	Enum	Open, Copy, Write, or Delete.
device.csv	date	DateTime	Device event timestamp.
device.csv	user	String	User identifier.
device.csv	activity	Enum	Connect or Disconnect.

Appendix B: Virtual Machine Deployment Specifications

This appendix provides the technical specifications of the virtual machine environments used for the deployment validation described in Chapters 3 and 4.

Specification	Windows VM	Ubuntu VM
Operating System	Windows 10 Pro (64-bit)	Ubuntu 22.04 LTS (64-bit)
vCPU Cores	4	8
RAM	8 GB	16 GB
Storage	100 GB (SSD)	200 GB (SSD)
Network	Internal Host-Only (192.168.x.x/24)	Internal Host-Only + NAT
Primary Role	Log Generation and Forwarding	Security Intelligence Platform
Key Software	Python 3.8, Filebeat 8.x	ELK Stack 8.x, Python 3.8, PyTorch 1.12